Vrattiks

DATABASE DESIGN DOCUMENT

Samaj Mobile Application

Vrattiks Database Design Document · PostgreSQL schema for community registration, consent-based matrimony, news, Panchang and the member directory

VERSION 1.0	AUTHOR Database Architecture, Vrattiks	TARGET ENGINE PostgreSQL 16
SCHEMA NAME samaj	DATE 2026-10-10	STATUS Draft for review

Contents

1 Executive Summary & Project Overview 2

2 Database Architecture & Environment 4

3 Data Models 6

4 Data Dictionary 17

5 Performance Optimization & Storage Strategy 35

6 Security, Compliance & Access Control 42

7 Operations, Backup & Disaster Recovery 46

8 Appendix 48

SOURCE DOCUMENTS

Samaj SOP v1.1 (09/10/26) · Samaj Mobile App PRD v1.0 (2026-10-09) · SRS VRT-SRS-SAMAJ-001 v1.0 (2026-10-09). **Stack:** React Native · FastAPI · SQLAlchemy 2.x · PostgreSQL 16.

Internal · Confidential — contains the physical design of systems that process personal data.

1 Executive Summary & Project Overview

Purpose & Scope

This document is the database blueprint for the **Samaj Mobile Application**: a React Native member app for iOS and Android and a web Admin Dashboard, both served by a FastAPI backend that uses SQLAlchemy over PostgreSQL. It turns the Samaj SOP (v1.1), PRD (v1.0) and SRS (VRT-SRS-SAMAJ-001 v1.0) into a physical schema that developers can migrate from, DBAs can provision from and QA can test against.

The design rests on one rule above all others: **shared biodata is visible only to User A and User B after User B accepts, never to any other member, City Admin or the Super Admin** (SOP §6.2, §8). That rule is enforced inside the database, not only in API code.

In scope: the transactional PostgreSQL database (schema `samaj`) for members, households, matrimony, biodata exchange, news and ads, Panchang, notifications, import/export, audit and DPDP rights. It also covers the read replica, roles, backups and the retention jobs. **Out of scope:** object storage contents (photos, ID documents and creatives are referenced by key only), the push/SMS/email providers' own stores, application logs, and features the SRS excludes: chat, payments, Kundli matching and a member web app.

Objectives

BIODATA EXPOSURE	DUPLICATE MEMBERS	READ QUERY TIME	MATRIMONY SEARCH
0	**0**	**< 50 ms P95**	**< 150 ms**
App roles cannot read the biodata table. Access goes only through a function that checks for an accepted pair.	Unique keyed hash of the E.164 mobile number, shared by registration and import (FR-05, FR-77)	DB time at 500 req/s peak, within the 500 ms API budget (NFR-02)	DB time with 4 filters over 50,000 profiles (NFR-04: ≤ 2 s end to end)

Target Audience

Reader	Uses this document to
Backend developers (FastAPI / SQLAlchemy)	Write SQLAlchemy models and Alembic migrations against the exact tables, keys and constraints in §4. Set the per-transaction session context that RLS and the biodata functions rely on (§6).
Mobile developers (React Native)	See which fields the API can ever return (public summary, directory fields, biodata only via the pair check) and the rule that biodata is never cached on the device (§2).
DBA / SRE	Provision the instance, PgBouncer, partitions, roles, backups and alerts (§2, §5, §7).
QA	Derive integrity and privacy tests from constraints. Examples: one open request per pair, 5 photos maximum, one City Admin per city, no biodata for third parties.
Security & data protection	Review PII handling, encryption, access control and DPDP Act 2023 obligations (§6).

SOURCE & ASSUMPTIONS

The source assumptions from SRS §2.6 still hold. The ones that shape the schema are: **ASM-01** about 10,000 members in about 10 cities, growing to 50,000; **ASM-02** Moderator and Content Admin duties belong to the City Admin; **ASM-03** "third party" includes all admins; **ASM-04** the mobile number is the unique identifier; **ASM-05** one Panchang location with a 04:00 IST cut-off; **ASM-08/09** a 30-day resend lock and 20 requests per day. This document adds:

- **ASM-11** Hosting on AWS in ap-south-1 (Mumbai) on Amazon RDS for PostgreSQL 16, keeping personal data in India. Azure Database for PostgreSQL Flexible Server in Central India is a drop-in alternative if OQ-06 resolves to Azure.
- **ASM-12** Background work (scheduler, push fan-out, Panchang fetch, imports, retention) runs in Python workers under separate database roles. The queue technology (OQ-11) does not affect this schema because notifications use a transactional outbox table.
- **ASM-13** Biodata is stored as one encrypted JSON document per profile, because the field list is not fixed yet (OQ-12).
- **ASM-14** About 15% of members create a matrimony profile, so about 7,500 profiles at 50,000 members. Each sends about 1 biodata request per week.
- **ASM-15** Identity documents are deleted 90 days after the review decision, as the SRS proposes (OQ-10).
- **ASM-16** OTP challenges and refresh-token sessions live in PostgreSQL, because the mandated stack has no cache store. They can move to Redis later without affecting the rest of the schema.
- **ASM-17** Eight tables are added beyond the SRS §6 entity list because the requirements need them: `household` (split from family member), `otp_challenge`, `auth_session`, `notification_outbox`, `import_row`, `layout_setting`, `panchang_revision` and `data_rights_request`.

2 Database Architecture & Environment

Platform & Version

PostgreSQL 16 on Amazon RDS. PostgreSQL is client-mandated (SRS CON-03). Version 16 meets the SRS floor of 15+ and is supported until November 2028. The design relies on features that make the business rules declarative:

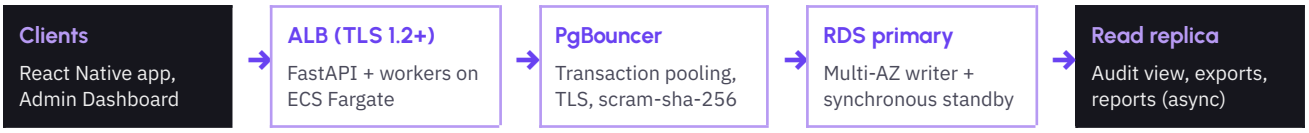
- partial unique indexes: one open biodata request per pair, one active City Admin per city;
- `SECURITY DEFINER` functions for the biodata pair check;
- row-level security for City Admin scope;
- declarative partitioning for logs, and a materialized view for search.

Extensions: `pgcrypto` (UUIDs, hashing), `citext` (admin e-mail), `pg_trgm` (directory name search), `pg_partman` (partition maintenance) and `pg_stat_statements` (query monitoring). All are supported on RDS.

Application Stack Integration

Layer	Database-relevant decision
FastAPI + SQLAlchemy 2.x	Async engine on `asyncpg`, one `AsyncSession` per request. Each transaction starts by setting `app.actor_role`, `app.member_id` / `app.admin_id`, `app.city_id` and `app.endpoint` with `set_config(..., true)`, which is transaction-local and so safe behind a transaction pooler. Pool per API task: `pool_size=5`, `max_overflow=5`, `pool_pre_ping=True`. Every statement has a 5 s `statement_timeout`.
Models & migrations	Alembic, versioned and reviewed (CON-02). Status fields are `VARCHAR` + `CHECK` rather than native PostgreSQL `ENUM`, so adding a value is a one-line migration. UUID keys come from `gen_random_uuid()` server defaults. Timestamps are `TIMESTAMPTZ` in UTC; the app displays IST.
Biodata access	The SQLAlchemy model for `biodata` is read-only metadata. Code calls `select(func.samaj.get_biodata(profile_id))` and decrypts the returned ciphertext. There is no ORM path that can select the table.
React Native app	Never connects to the database. It may cache public data (Panchang, news, its own profile) for offline use. Biodata responses carry `Cache-Control: no-store` and are never written to AsyncStorage or MMKV.
Workers	Poll `notification_outbox` with `FOR UPDATE SKIP LOCKED`. They run the scheduler (content go-live and expiry, the 4-hourly search refresh, Panchang fetch) and the import processor under their own DB role.

Hosting Environment & Connectivity



The database endpoints sit in private subnets with no public access. Only the PgBouncer security group can reach the primary on port 5432. Senior technical staff reach a read-only role through the VPN and SSO (SOP §4, NFR-17), and every session is logged. PgBouncer runs in transaction mode with `max_prepared_statements` enabled, so asyncpg's prepared statements work. Staging mirrors production at a smaller instance class (SOP §9).

Hardware & Capacity

Component	Configuration	Rationale
Primary	db.m7g.large (2 vCPU, 8 GiB RAM), Multi-AZ	The data is small: under 5 GiB with indexes at 50,000 members and 24 months of logs. So the instance is sized for 500 req/s peak and fast failover, not for storage. The working set fits in memory.
Read replica	db.t4g.large (2 vCPU, 8 GiB), same region	Keeps the audit view, CSV downloads and filtered exports off the writer.
Storage	gp3, 50 GiB, 3,000 IOPS baseline, 125 MiB/s, autoscaling to 200 GiB	Year-2 estimate is about 5 GiB. Headroom covers WAL, bloat and index rebuilds (SOP §11 monthly optimisation).
Connections	`max_connections` 200. PgBouncer server pool of 40 for the API and 15 for workers and jobs.	4–8 API tasks × 10 client connections, multiplexed onto 40 server connections. Leaves room for migrations and ops.
Key settings	`shared_buffers` 2 GiB, `work_mem` 16 MiB, `idle_in_transaction_session_timeout` 30 s, `rds.force_ssl` 1, `timezone` UTC	Defaults tuned for a small OLTP workload. Idle transactions cannot hold locks for long.

SCALE-UP TRIGGER

Move to db.m7g.xlarge if any of these hold: CPU above 65% for 30 minutes at peak; buffer cache hit ratio below 99% for a week; or P95 read query time above 50 ms (from `pg_stat_statements`). The jump from ASM-01 to NFR-08 (50 cities, 50,000 members) does not need sharding.

3 Data Models

Conceptual Data Model

Members belong to a **City** and a **Household**. A member may list one **Matrimony Profile**, whose private **Biodata** is unlocked only by an accepted **Biodata Request** between two profiles. **Admins** act within a city: they approve members, moderate photos, publish **Content**, maintain the **Panchang** and run imports and exports. Every one of those actions is recorded in the **Audit Log**.

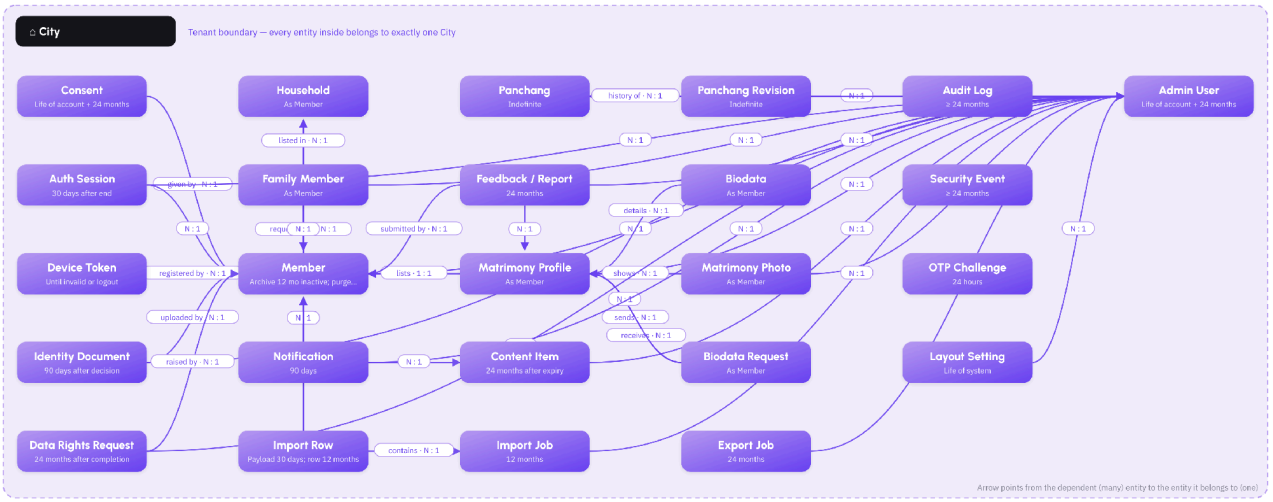


Figure 1 · Conceptual data model — business entities and how they relate

Logical Data Model

Table	Entity & purpose	Est. volume	Retention	Sensitivity
`city`	**City** A city served by the deployment. Every member, household, City Admin and city-targeted content item belongs to exactly one city.	~10 rows → ≥ 50	Life of system	Low
`admin_user`	**Admin User** A Super Admin or City Admin who signs in to the Admin Dashboard with password + TOTP. Exactly one active City Admin per city (FR-70).	~10–60 rows	Life of account + 24 months	● High
`household`	**Household** A family unit in the directory. Members link to it; approved non-app relatives are family_member rows under it.	~4k → 20k rows	As Member	● High
`member`	**Member** A registered app user, identified by a unique mobile number. Lifecycle: PENDING → ACTIVE / REJECTED → ARCHIVED → purged (anonymised).	~10k → 50k rows	Archive 12 mo inactive; purge 24 mo	● High
`family_member`	**Family Member** A relative added to a household through a family-addition request (FR-17). Visible in the directory only once APPROVED.	~30k → 150k rows	As Member	● High

Table	Entity & purpose	Est. volume	Retention	Sensitivity
`identity_document`	**Identity Document** Optional ID uploaded at registration (FR-09). The file sits in a private, KMS-encrypted bucket; this row holds only its key and metadata.	~10k rows	90 days after decision	● High
`consent`	**Consent** Purpose-specific DPDP consent captured before a profile is created (FR-07). Append-style: a new notice version creates a new row.	~1–2 rows per member	Life of account + 24 months	Medium
`otp_challenge`	**OTP Challenge** A one-time password sent for login, registration or erasure confirmation. Enforces 5-minute expiry, 3 attempts and 5 sends/hour (FR-03, FR-04).	~30k rows/month (transient)	24 hours	Medium
`auth_session`	**Auth Session** A refresh-token session for a member (30-day rotating refresh, NFR-16) or an admin (30-min idle timeout, FR-73). Exactly one principal per row.	~60k active rows	30 days after end	Medium
`device_token`	**Device Token** An FCM registration token for a member's device (FR-53). Marked invalid when FCM reports UNREGISTERED (FR-55).	~15k → 70k rows	Until invalid or logout	Medium
`matrimony_profile`	**Matrimony Profile** The public summary of a member's matrimony listing (FR-08, FR-20). Contains no biodata — that lives in the separately protected biodata table.	~1.5k → 7.5k rows (ASM)	As Member	Medium
`biodata`	**Biodata** The private biodata of a matrimony profile, stored as one AES-256-GCM encrypted JSON document. Readable only through samaj.get_biodata(), which enforces the accepted-pair rule.	1 row per matrimony profile	As Member	● High
`matrimony_photo`	**Matrimony Photo** A photo on a matrimony profile, moderated against community guidelines before it is shown (FR-09, FR-23). Maximum 5 per profile.	~5k → 35k rows	As Member	Medium
`biodata_request`	**Biodata Request** User A's request to exchange biodata with User B (SOP §6.2). ACCEPTED grants mutual access to that pair only. At most one PENDING or ACCEPTED request per pair, in either direction (FR-30).	~20k → 100k rows/year (ASM)	As Member	● High
`content_item`	**Content Item** A news item or ad campaign scheduled in Content Management (SOP §6.3). Goes live at start_at and expires at end_at automatically (FR-47).	~3k rows/year	24 months after expiry	Low
`layout_setting`	**Layout Setting** Key/value configuration for ad dimensions and feed placement (FR-50, FR-51), e.g. ad_every_n_items = 5.	< 50 rows	Life of system	Low

Table	Entity & purpose	Est. volume	Retention	Sensitivity
`panchang_day`	**Panchang** The effective Tithi, Nakshatra and Yoga for one date and reference location (SOP §6.4). A manual entry overrides API values (FR-62).	365 rows/year per location	Indefinite	Low
`panchang_revision`	**Panchang Revision** Append-only history of every API fetch and manual edit of a Panchang day: the edit history shown in the dashboard (FR-62).	~400–800 rows/year	Indefinite	Low
`notification_outbox`	**Notification** Transactional outbox for push / SMS / email. Written in the same transaction as the business change, then delivered by the worker (≤ 30 s target, NFR-05).	~20k–60k rows/month	90 days	Medium
`import_job`	**Import Job** A CSV/XLSX member-list upload (SOP §6.5). Validated → dry-run report → committed. A header mismatch rejects the file before any write (FR-76).	~500 rows/year	12 months	Medium
`import_row`	**Import Row** One validated row of an import, with its dedup outcome (NEW / UPDATE / CONFLICT / ERROR). Staged data is encrypted and wiped 30 days after commit.	~10k rows per import	Payload 30 days; row 12 months	● High
`export_job`	**Export Job** Record of an 'Export Filtered' (FR-82). The password and the file are never stored; only who exported what, and how many rows.	~1k rows/year	24 months	Medium
`audit_log`	**Audit Log** Append-only record of every admin action and data-principal event (FR-86, FR-87). No FKs, so log rows outlive the objects they describe.	~20k–50k rows/month	≥ 24 months	● High
`security_event`	**Security Event** Denied or suspicious access, above all every denied biodata read (FR-37), plus OTP lockouts, MFA failures (ERR_AUTH_01) and cross-city 403s.	~1k–5k rows/month	≥ 24 months	● High
`feedback_report`	**Feedback / Report** In-app feedback, a profile report (FR-91) or a DPDP grievance (FR-100), reviewed daily in the dashboard.	~2k rows/year	24 months	Medium
`data_rights_request`	**Data Rights Request** A DPDP data-principal request: access copy, erasure (OTP-confirmed) or consent withdrawal, which triggers erasure. Completed within 30 days (FR-98, FR-99).	~100s rows/year	24 months after completion	Medium

Every entity in SRS §6.1 maps to a table, with its retention and sensitivity carried over:

SRS entity	Table(s)	Note
City	`city`	The SRS `city_admin_id` becomes `admin_user.city_id` plus a partial unique index, which avoids a circular FK and keeps the one-admin-per-city rule declarative.
AdminUser	`admin_user`	Argon2id password hash and encrypted TOTP secret, as specified.

SRS entity	Table(s)	Note
Member	**member**	The mobile number is split into `mobile_enc` (encrypted), `mobile_hash` (unique lookup) and `mobile_last4` (masked display).
Household / FamilyMember	**household, family_member**	`head_member_id` becomes `member.is_household_head` with one head per household.
IdentityDocument	**identity_document**	Metadata only; the file sits in a private, KMS-encrypted bucket.
Consent	**consent**	One row per purpose and notice version.
MatrimonyProfile · Biodata · MatrimonyPhoto	**matrimony_profile, biodata, matrimony_photo**	Public summary and biodata live in separate tables with separate access rules (FR-20).
BiodataRequest	**biodata_request**	Links two matrimony profiles. The generated `pair_low`/`pair_high` columns make the pair direction-independent.
ContentItem	**content_item**	News and ads in one table; a NULL `city_id` means all cities.
Panchang	**panchang_day, panchang_revision**	The effective values plus append-only edit history (FR-62).
DeviceToken	**device_token**	One row per FCM token.
ImportJob · ExportJob	**import_job, import_row, export_job**	Row-level dry-run results are needed for the conflict review (FR-78, FR-79).
AuditLog · SecurityEvent	**audit_log, security_event**	Partitioned monthly and append-only.
Feedback / Report	**feedback_report**	Also holds DPDP grievances (FR-100).

The physical ERD shows every table with its keys. References to the widely shared `city` and `admin_user` tables appear as outlined FK tags rather than lines, to keep the diagram readable. `audit_log` and `security_event` deliberately have no FKs, so log rows outlive the objects they describe.

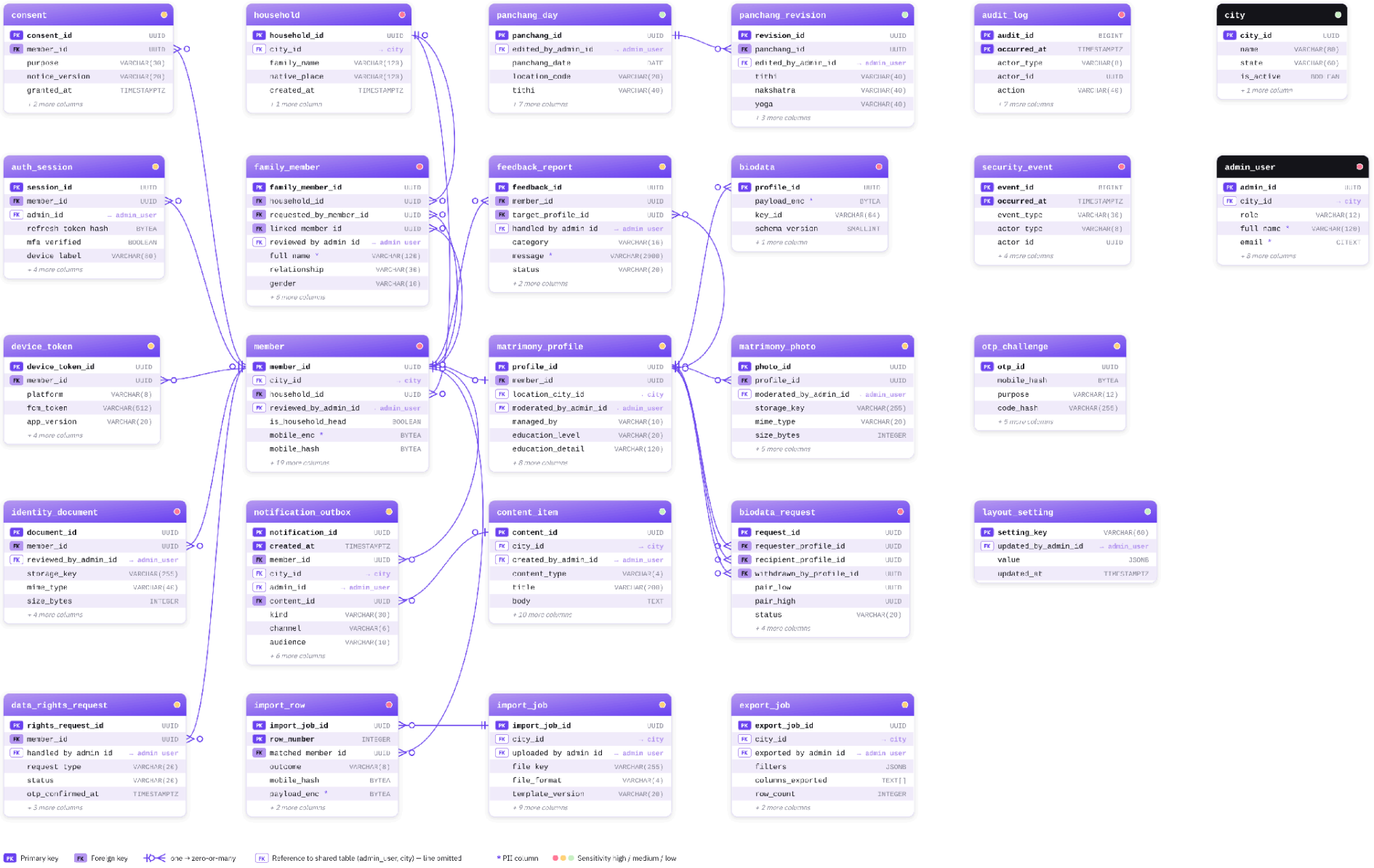


Figure 2 · Physical entity-relationship diagram (crow's-foot notation)

Relationships & Cardinality

Parent (one)	Cardinality	Child (many)	Meaning	On delete
city	1 : N (tenant)	admin_user, content_item, export_job, household, import_job, matrimony_profile, member, notification_outboxvia city_id, location_city_id on each	Tenant ownership — every row belongs to one tenant	RESTRICT
household	1 : N (optional)	membervia household_id	belongs to	SET NULL
admin_user	1 : N (optional)	membervia reviewed_by_admin_id	reviewed by	SET NULL
household	1 : N (mandatory)	family_membervia household_id	listed in	CASCADE
member	1 : N (mandatory)	family_membervia requested_by_member_id	requested by	RESTRICT
member	1 : N (optional)	family_membervia linked_member_id		SET NULL
admin_user	1 : N (optional)	family_membervia reviewed_by_admin_id		SET NULL
member	1 : N (mandatory)	identity_documentvia member_id	uploaded by	CASCADE
admin_user	1 : N (optional)	identity_documentvia reviewed_by_admin_id		SET NULL
member	1 : N (mandatory)	consentvia member_id	given by	RESTRICT
member	1 : N (optional)	auth_sessionvia member_id		CASCADE
admin_user	1 : N (optional)	auth_sessionvia admin_id		CASCADE
member	1 : N (mandatory)	device_tokenvia member_id	registered by	CASCADE
member	1 : 1 (mandatory)	matrimony_profilevia member_id	lists	RESTRICT
admin_user	1 : N (optional)	matrimony_profilevia moderated_by_admin_id		SET NULL
matrimony_profile	1 : N (mandatory)	biodatavia profile_id	details	CASCADE
matrimony_profile	1 : N (mandatory)	matrimony_photovia profile_id	shows	CASCADE
admin_user	1 : N (optional)	matrimony_photovia moderated_by_admin_id		SET NULL
matrimony_profile	1 : N (mandatory)	biodata_requestvia requester_profile_id	sends	CASCADE
matrimony_profile	1 : N (mandatory)	biodata_requestvia recipient_profile_id	receives	CASCADE
matrimony_profile	1 : N (optional)	biodata_requestvia withdrawn_by_profile_id		SET NULL
admin_user	1 : N (mandatory)	content_itemvia created_by_admin_id		RESTRICT
admin_user	1 : N (optional)	layout_settingvia updated_by_admin_id		SET NULL
admin_user	1 : N (optional)	panchang_dayvia edited_by_admin_id		SET NULL
panchang_day	1 : N (mandatory)	panchang_revisionvia panchang_id	history of	RESTRICT
admin_user	1 : N (optional)	panchang_revisionvia edited_by_admin_id		SET NULL
member	1 : N (optional)	notification_outboxvia member_id		CASCADE

Parent (one)	Cardinality	Child (many)	Meaning	On delete
admin_user	1 : N (optional)	notification_outbox via admin_id		RESTRICT
content_item	1 : N (optional)	notification_outbox via content_id		SET NULL
admin_user	1 : N (mandatory)	import_job via uploaded_by_admin_id		RESTRICT
import_job	1 : N (mandatory)	import_row via import_job_id	contains	CASCADE
member	1 : N (optional)	import_row via matched_member_id		SET NULL
admin_user	1 : N (mandatory)	export_job via exported_by_admin_id		RESTRICT
member	1 : N (optional)	feedback_report via member_id	submitted by	SET NULL
matrimony_profile	1 : N (optional)	feedback_report via target_profile_id		SET NULL
admin_user	1 : N (optional)	feedback_report via handled_by_admin_id		SET NULL
member	1 : N (mandatory)	data_rights_request via member_id	raised by	RESTRICT
admin_user	1 : N (optional)	data_rights_request via handled_by_admin_id		SET NULL

Normalization Strategy

The schema is in **Third Normal Form**. Two places hold variable data by design. Biodata is one encrypted document (ASM-13), because encryption at field level rules out querying it anyway. Settings use typed JSONB values. The following denormalisations are deliberate:

Denormalised field	Why	How it stays correct
member.mobile_last4	Masked display ("••••••4321") without decrypting.	Written together with `mobile_enc` and wiped on erasure.
biodata_request.pair_low / pair_high	Direction-independent uniqueness and pair lookup.	`GENERATED ALWAYS … STORED` from the two profile IDs, so they cannot drift.
matrimony_search_mv	Fast filtered search, and structural proof that search can never return biodata: the view does not contain it.	Refreshed every ≤ 4 h (SOP §6.2) and right after any erasure. The search query re-joins `matrimony_profile.status`, so profiles hidden since the last refresh drop out at once.
household.city_id	City Admin scope for households without a join through members.	Set from the head member's city. A city move updates the member and household in one transaction.
import_job.*_count	The dashboard shows totals without counting `import_row`.	Written by the worker in the same transaction that classifies the rows.
audit_log.city_id	The audit view's city filter (FR-88) on a table with no FKs.	Copied from the target row when the entry is written. Log rows are immutable.

Data Flows

How each SOP journey and background job touches the tables, in transaction order. Database reads appear above each step and writes below it.

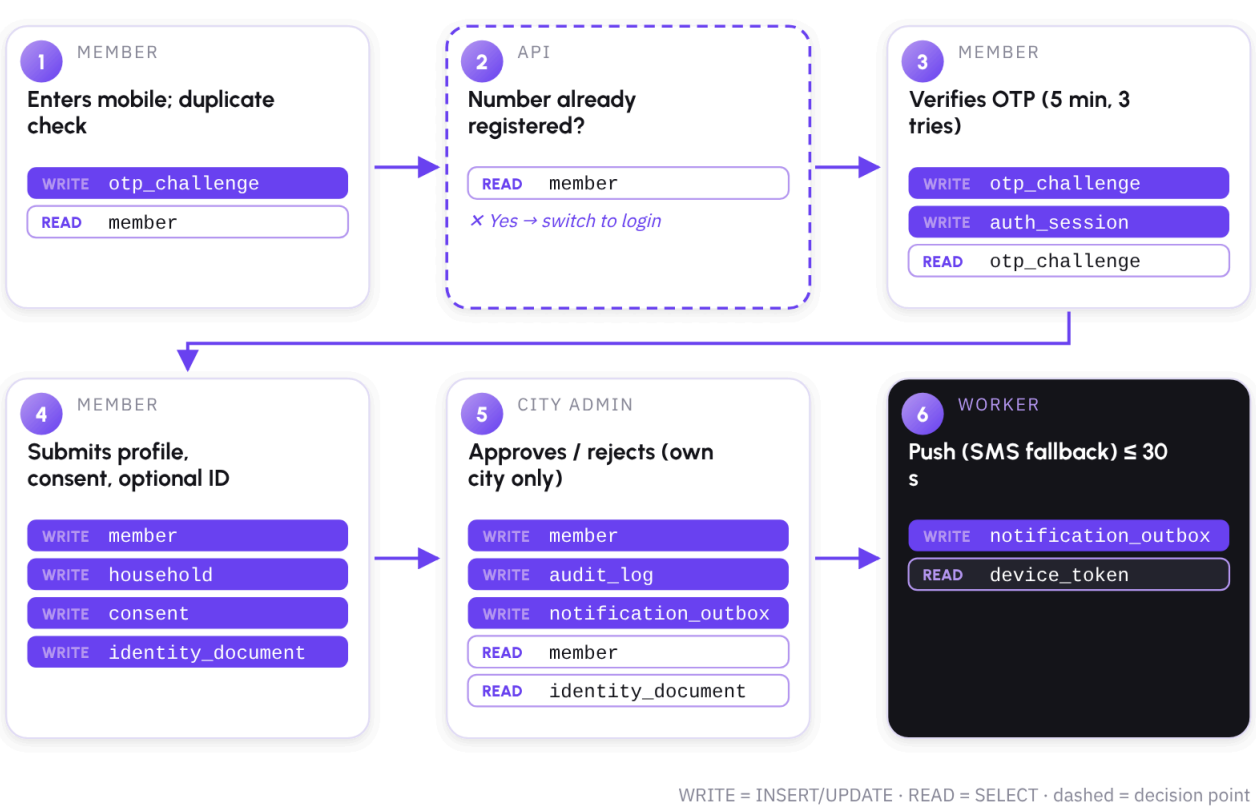


Figure 3 · Data flow — Registration, OTP and City Admin approval (SOP §6.1)

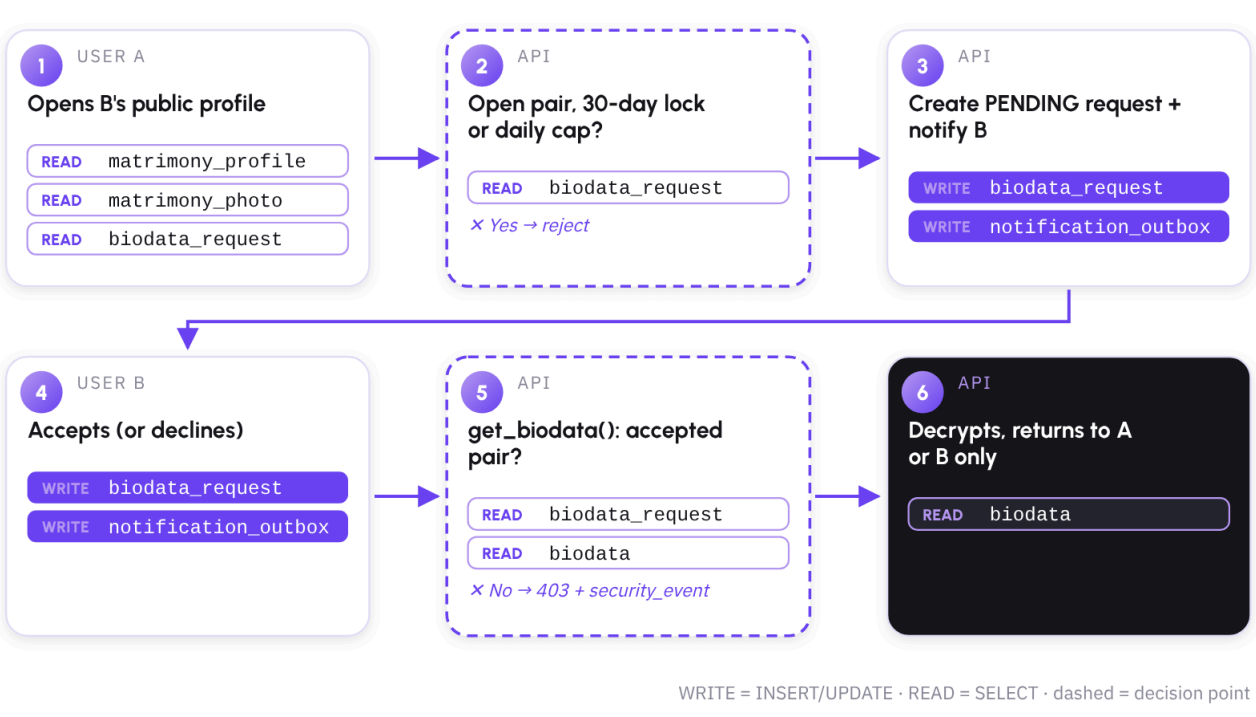


Figure 4 · Data flow — Biodata request and consent-based exchange (SOP §6.2)

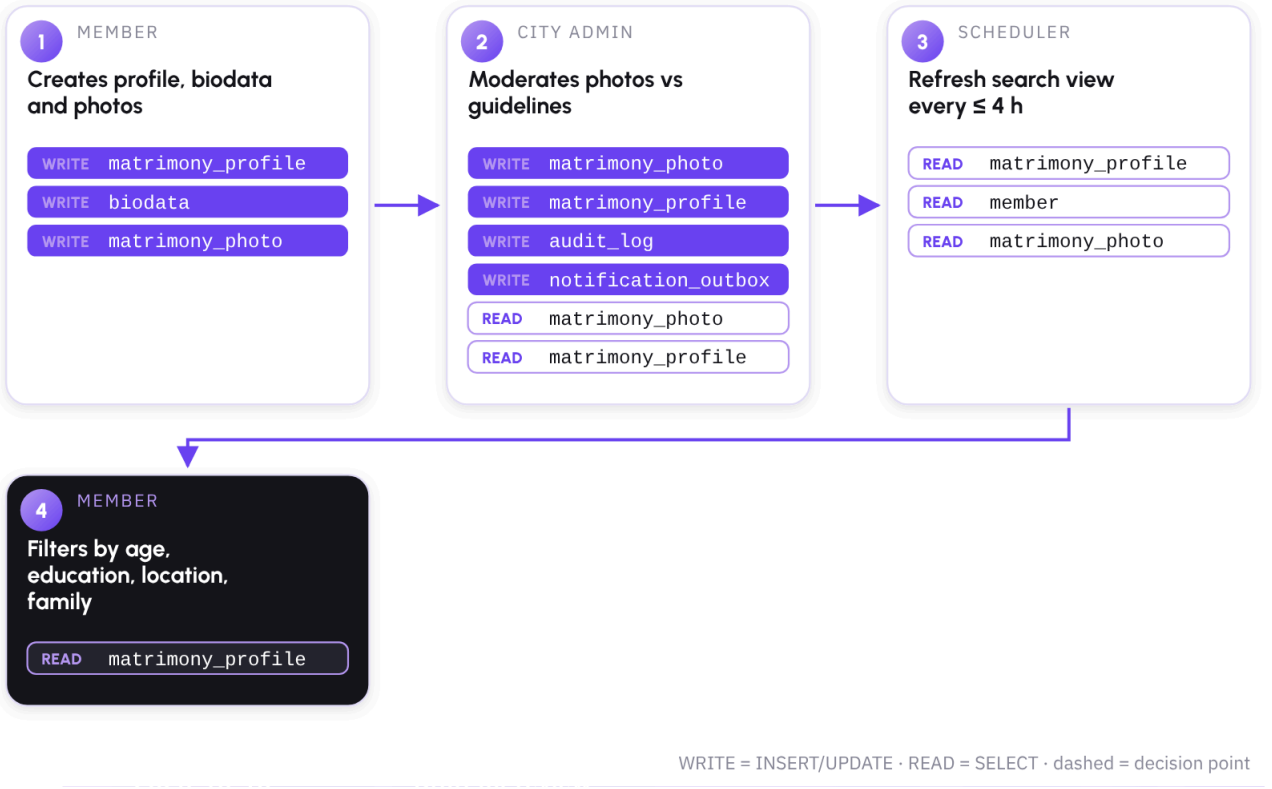


Figure 5 · Data flow — Matrimony listing, moderation and search refresh (SOP §6.2)

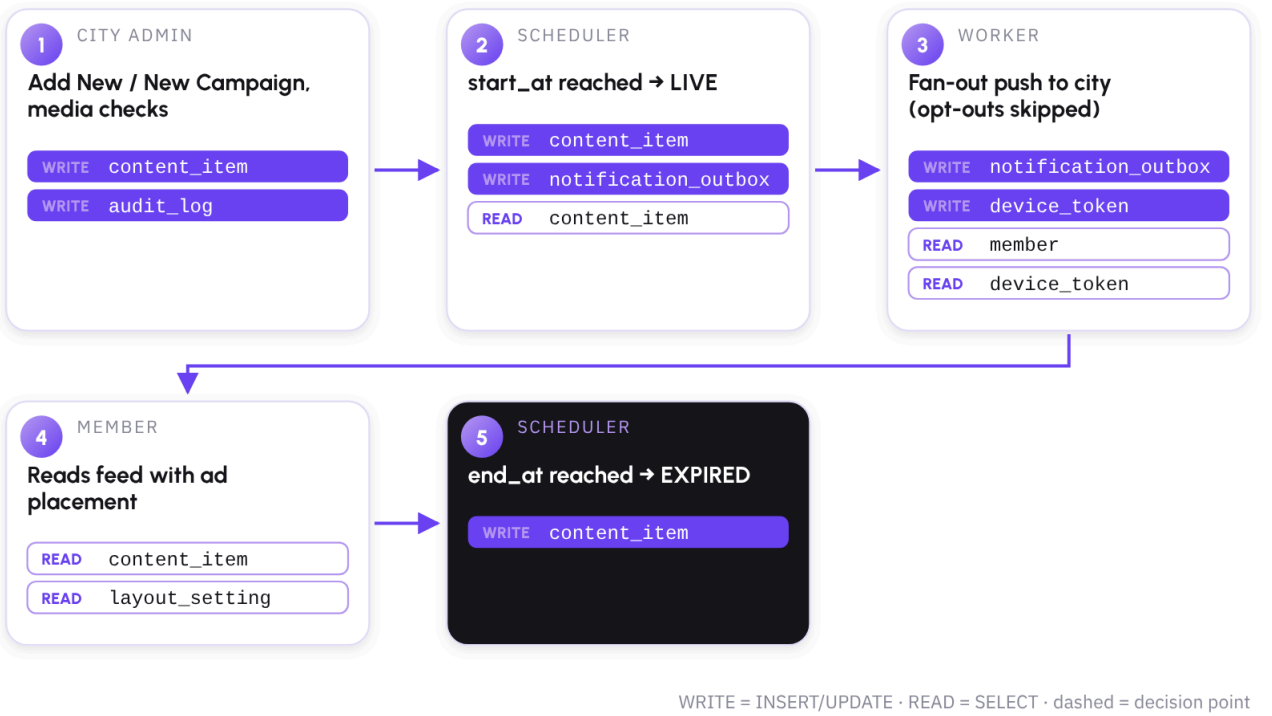


Figure 6 · Data flow — News and ads scheduling (SOP §6.3)

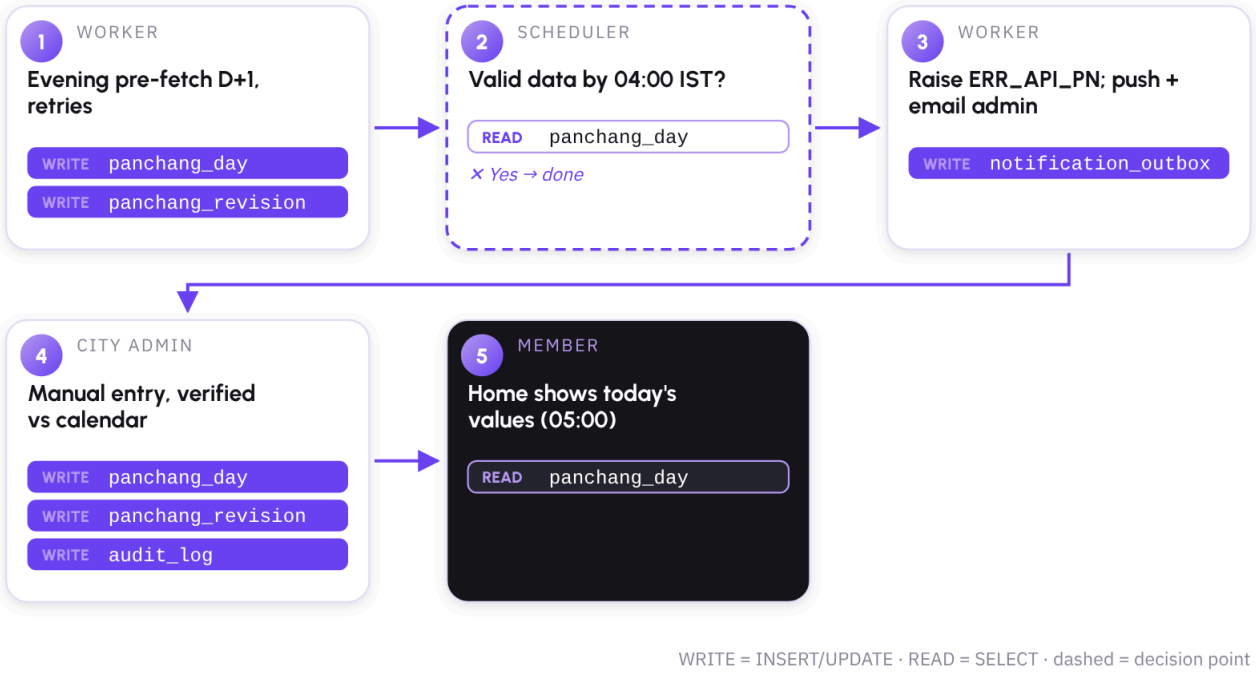


Figure 7 · Data flow — Daily Panchang with manual fallback (SOP §6.4, ERR_API_PN)

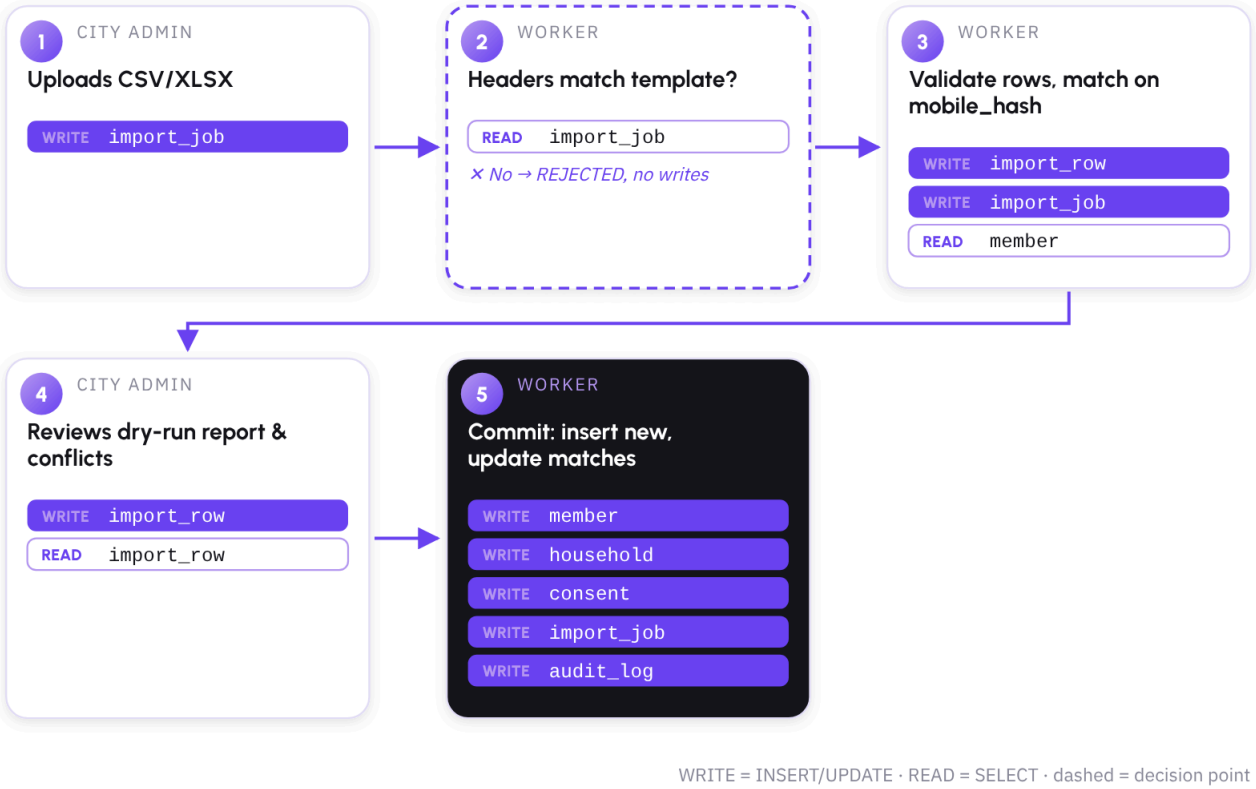


Figure 8 · Data flow — Weekly member import with deduplication (SOP §6.5)

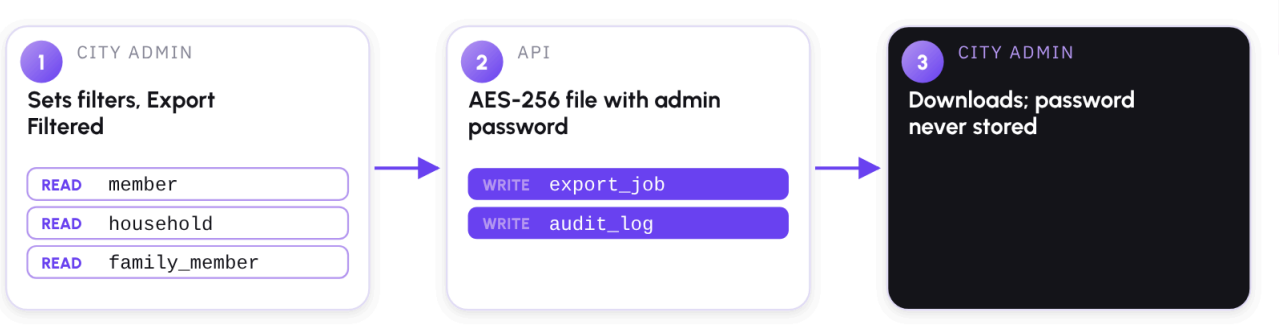


Figure 9 · Data flow — Encrypted filtered export (SOP §6.5)

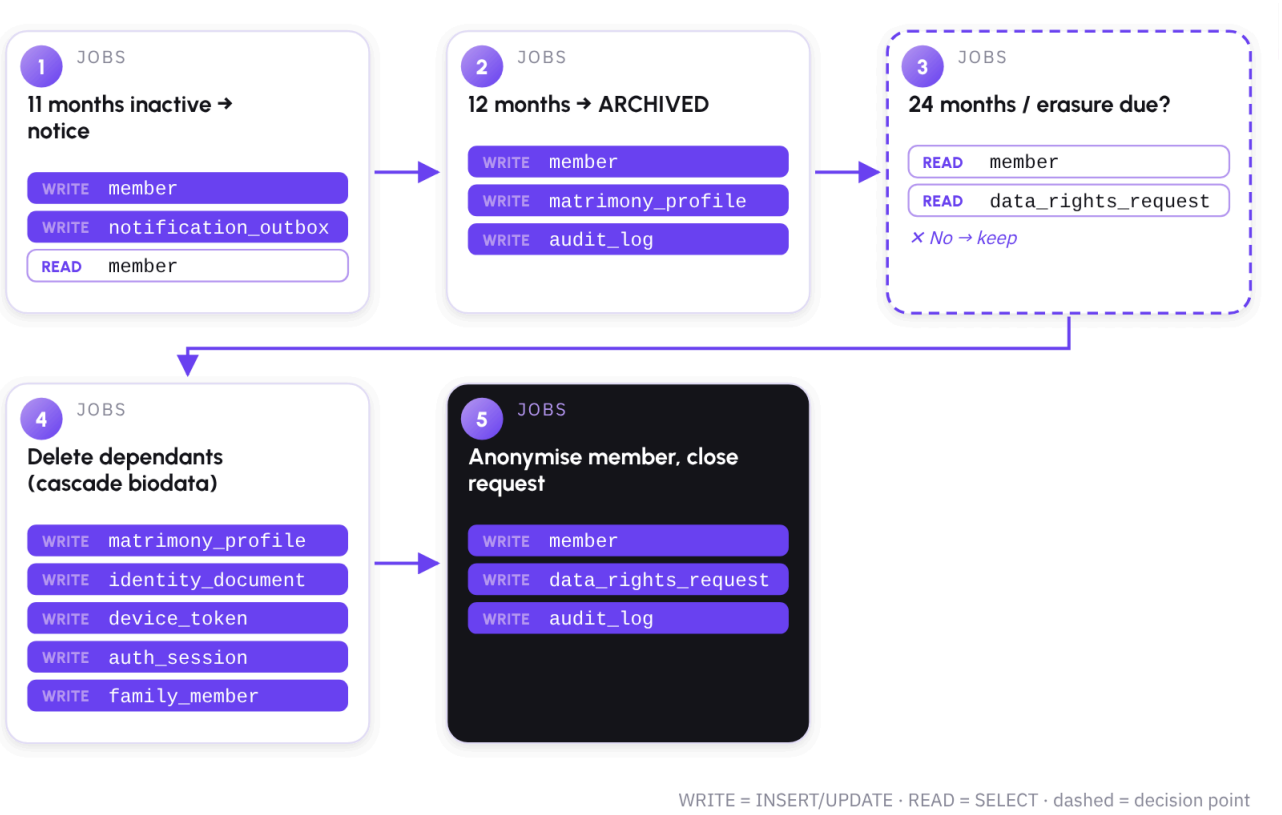


Figure 10 · Data flow — Retention, archive, erasure and recovery jobs (SOP §8, §12)

4 Data Dictionary

Physical specification of every table in schema `samaj`. Conventions:

- snake_case, singular table names, with `<table>_id` UUID primary keys;
- every timestamp is `TIMESTAMPTZ` in UTC; the Panchang date is an IST calendar `DATE`;
- columns marked PII are personal data under the DPDP Act 2023;
- columns with an encryption badge are encrypted by the API before they reach the database.

samaj.city City

Retention · Life of system | Sensitivity · Low | Volume · ~10 rows → ≥ 50

A city served by the deployment. Every member, household, City Admin and city-targeted content item belongs to exactly one city.

Field name	Data type	Constraints	Key	Description
city_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
name	VARCHAR(80)	NOT NULL		City name as shown in the app's city picker.
state	VARCHAR(60)	NOT NULL		Indian state or union territory.
is_active	BOOLEAN	NOT NULL DEFAULT true		False hides the city from registration and content targeting.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

Composite unique: `(name, state)`

samaj.admin_user Admin User

Retention · Life of account + 24 months | Sensitivity · High | Volume · ~10–60 rows

A Super Admin or City Admin who signs in to the Admin Dashboard with password + TOTP. Exactly one active City Admin per city (FR-70).

Field name	Data type	Constraints	Key	Description
admin_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
role	VARCHAR(12)	NOT NULL CHECK (role IN ('SUPER_ADMIN','CITY_ADMIN'))		SUPER_ADMIN or CITY_ADMIN (SOP §3).
city_id	UUID	NULL ON DELETE RESTRICT	FK → city IDX	Assigned city; required for CITY_ADMIN, NULL for SUPER_ADMIN.
full_name	VARCHAR(120)	NOT NULL		PII Admin's display name.
email	CITEXT	NOT NULL UNIQUE	UQ	PII Login identifier (case-insensitive).
password_hash	VARCHAR(255)	NOT NULL		Argon2id hash of the dashboard password; the password itself is never stored.
totp_secret_enc	BYTEA	NULL		AES-256-GCM TOTP shared secret, AES-256-GCM encrypted by the API with a KMS data key.

Field name	Data type	Constraints	Key	Description
mfa_enrolled_at	TIMESTAMPTZ	NULL		When TOTP enrolment completed; NULL forces re-enrolment (FR-74).
status	VARCHAR(20)	NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','SUSPENDED','DEACTIVATED'))		Account state; only ACTIVE admins can sign in.
last_login_at	TIMESTAMPTZ	NULL		Last successful MFA login.
deactivated_at	TIMESTAMPTZ	NULL		Retention clock for anonymisation.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

Table check: `(role = 'SUPER_ADMIN' AND city_id IS NULL) OR (role = 'CITY_ADMIN' AND city_id IS NOT NULL)`

samaj.household Household

Retention · As Member | Sensitivity · High | Volume · ~4k → 20k rows

A family unit in the directory. Members link to it; approved non-app relatives are family_member rows under it.

Field name	Data type	Constraints	Key	Description
household_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
city_id	UUID	NOT NULL ON DELETE RESTRICT	FK → city IDX	City the household lives in; drives City Admin scope.
family_name	VARCHAR(120)	NOT NULL		Surname / family name used for directory search (FR-65).
native_place	VARCHAR(120)	NULL		Ancestral village or town (optional directory field).
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

samaj.member Member

Retention · Archive 12 mo inactive; purge 24 mo | Sensitivity · High | Volume · ~10k → 50k rows

A registered app user, identified by a unique mobile number. Lifecycle: PENDING → ACTIVE / REJECTED → ARCHIVED → purged (anonymised).

Field name	Data type	Constraints	Key	Description
member_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key; the only member identifier used in other tables, logs and audit.
city_id	UUID	NOT NULL ON DELETE RESTRICT	FK → city IDX	Home city; routes the registration to that city's admin (FR-11).
household_id	UUID	NULL ON DELETE SET NULL	FK → household IDX	Household the member belongs to; NULL until set.
is_household_head	BOOLEAN	NOT NULL DEFAULT false		True for the head of the household (one per household).

Field name	Data type	Constraints	Key	Description
mobile_enc	BYTEA	NULL		PII AES-256-GCM E.164 mobile number (+91XXXXXXXXXX), AES-256-GCM encrypted by the API.
mobile_hash	BYTEA	NOT NULL UNIQUE	UQ	HMAC-SHA256(pepper, E.164 mobile). Unique; used for login, duplicate check and import dedup (FR-05, FR-77).
mobile_last4	CHAR(4)	NULL		PII Last four digits for masked display without decryption.
full_name	VARCHAR(120)	NULL		PII Member's full name; searchable in the directory.
gender	VARCHAR(10)	NULL CHECK (gender IN ('MALE','FEMALE','OTHER'))		Gender as declared.
dob	DATE	NULL		PII Date of birth; drives matrimony age filter and import secondary match.
address_enc	BYTEA	NULL		PII AES-256-GCM Postal address, AES-256-GCM encrypted.
status	VARCHAR(20)	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING','ACTIVE','REJECTED','ARCHIVED'))	IDX	Account state. PENDING sees Panchang & News only (ASM-07).
rejection_reason	VARCHAR(500)	NULL		Shown to the member when REJECTED (FR-13).
reviewed_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	City Admin who approved / rejected.
reviewed_at	TIMESTAMPTZ	NULL		Approval / rejection time; source of the turnaround KPI.
hide_phone	BOOLEAN	NOT NULL DEFAULT false		Hide phone number from directory results (FR-68).
hide_address	BOOLEAN	NOT NULL DEFAULT false		Hide address from directory results (FR-68).
notify_news	BOOLEAN	NOT NULL DEFAULT true		Receive news / ad pushes (FR-56). Account and biodata pushes are always on.
source	VARCHAR(10)	NOT NULL DEFAULT 'APP' CHECK (source IN ('APP','IMPORT'))		How the row was created.
last_login_at	TIMESTAMPTZ	NULL	IDX	Last successful OTP login; retention clock (FR-93).
archive_notice_at	TIMESTAMPTZ	NULL		When the 30-day archive warning was sent (FR-92).
archived_at	TIMESTAMPTZ	NULL		When the member was archived for inactivity.
deleted_at	TIMESTAMPTZ	NULL	IDX	Soft delete; recoverable for 30 days via Archive Recovery (FR-96).
erased_at	TIMESTAMPTZ	NULL		Set when PII was purged/anonymised; row kept for referential integrity.

Field name	Data type	Constraints	Key	Description
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	IDX	Row creation time (UTC).
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

Table check: `status <> 'REJECTED' OR rejection_reason IS NOT NULL`

Table check: `dob <= CURRENT_DATE`

Name and DOB stay plaintext (covered by storage encryption) because the directory and matrimony age filters query them; phone and address are field-encrypted because they are only displayed, never filtered.

samaj.family_member Family Member

Retention · As Member | Sensitivity · High | Volume · ~30k → 150k rows

A relative added to a household through a family-addition request (FR-17). Visible in the directory only once APPROVED.

Field name	Data type	Constraints	Key	Description
family_member_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
household_id	UUID	NOT NULL ON DELETE CASCADE	FK → household IDX	Household the relative is added to.
requested_by_member_id	UUID	NOT NULL ON DELETE RESTRICT	FK → member IDX	Member who submitted the addition request.
linked_member_id	UUID	NULL ON DELETE SET NULL	FK → member IDX	Set when the relative is also a registered member.
full_name	VARCHAR(120)	NOT NULL	PII	Relative's name.
relationship	VARCHAR(30)	NOT NULL		Relationship to the household head (e.g. SPOUSE, SON, MOTHER).
gender	VARCHAR(10)	NULL CHECK (gender IN ('MALE','FEMALE','OTHER'))		Gender.
dob	DATE	NULL	PII	Date of birth.
status	VARCHAR(20)	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING','APPROVED','REJECTED'))	IDX	Approval state (FR-18).
rejection_reason	VARCHAR(500)	NULL		Shown to the requester on rejection.
reviewed_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	City Admin who decided.
reviewed_at	TIMESTAMPTZ	NULL		Decision time.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	IDX	Row creation time (UTC).

Table check: `status <> 'REJECTED' OR rejection_reason IS NOT NULL`

samaj.identity_document Identity Document

Retention · 90 days after decision | Sensitivity · High | Volume · ~10k rows

Optional ID uploaded at registration (FR-09). The file sits in a private, KMS-encrypted bucket; this row holds only its key and metadata.

Field name	Data type	Constraints	Key	Description
document_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
member_id	UUID	NOT NULL ON DELETE CASCADE	FK → member IDX	Member who uploaded it.
storage_key	VARCHAR(255)	NOT NULL		Object key in the private bucket; never a public URL. Served via ≤ 15-min pre-signed URL.
mime_type	VARCHAR(40)	NOT NULL CHECK (mime_type IN ('image/jpeg','image/png', 'application/pdf'))		image/jpeg, image/png or application/pdf.
size_bytes	INTEGER	NOT NULL CHECK (size_bytes BETWEEN 1 AND 2097152)		File size; ≤ 2 MB (ERR_IMG_UL).
sha256	CHAR(64)	NOT NULL		Content hash for integrity and duplicate-upload detection.
reviewed_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	Admin who reviewed the document with the registration.
reviewed_at	TIMESTAMPTZ	NULL		Review decision time; retention clock.
purge_after	TIMESTAMPTZ	NULL	IDX	Set to reviewed_at + 90 days; the purge job deletes when passed.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

samaj.consent Consent

Retention · Life of account + 24 months | Sensitivity · Medium | Volume · ~1–2 rows per member

Purpose-specific DPDP consent captured before a profile is created (FR-07). Append-style: a new notice version creates a new row.

Field name	Data type	Constraints	Key	Description
consent_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
member_id	UUID	NOT NULL ON DELETE RESTRICT	FK → member IDX	Data principal who consented.
purpose	VARCHAR(30)	NOT NULL CHECK (purpose IN ('MEMBERSHIP','MATRIMONY', 'DIRECTORY','NOTIFICATIONS'))		Processing purpose consented to.
notice_version	VARCHAR(20)	NOT NULL		Version of the consent notice shown (e.g. 2026.1).
granted_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		When consent was given.
withdrawn_at	TIMESTAMPTZ	NULL		When consent was withdrawn; triggers the erasure workflow (FR-99).

Field name	Data type	Constraints	Key	Description
channel	VARCHAR(10)	NOT NULL DEFAULT 'APP' CHECK (channel IN ('APP','IMPORT'))		Where consent was captured.

Composite unique: `(member_id, purpose, notice_version)`

samaj.otp_challenge OTP Challenge

Retention · 24 hours | Sensitivity · Medium | Volume · ~30k rows/month (transient)

A one-time password sent for login, registration or erasure confirmation. Enforces 5-minute expiry, 3 attempts and 5 sends/hour (FR-03, FR-04).

Field name	Data type	Constraints	Key	Description
otp_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
mobile_hash	BYTEA	NOT NULL	IDX	HMAC of the target mobile; no plaintext number is stored here.
purpose	VARCHAR(12)	NOT NULL CHECK (purpose IN ('LOGIN','REGISTER','ERASURE'))		Why the OTP was sent.
code_hash	VARCHAR(255)	NOT NULL		Argon2id hash of the 6-digit code.
attempts	SMALLINT	NOT NULL DEFAULT 0 CHECK (attempts BETWEEN 0 AND 3)		Failed verification attempts; locked at 3.
expires_at	TIMESTAMPTZ	NOT NULL		created_at + 5 minutes.
consumed_at	TIMESTAMPTZ	NULL		Set on successful verification; a consumed OTP cannot be reused.
request_ip	INET	NULL		Caller IP for the 10 req/min/IP limit (NFR-18).
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	IDX	Row creation time (UTC).

Table check: `expires_at > created_at`

samaj.auth_session Auth Session

Retention · 30 days after end | Sensitivity · Medium | Volume · ~60k active rows

A refresh-token session for a member (30-day rotating refresh, NFR-16) or an admin (30-min idle timeout, FR-73). Exactly one principal per row.

Field name	Data type	Constraints	Key	Description
session_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
member_id	UUID	NULL ON DELETE CASCADE	FK → member IDX	Member principal (NULL for admin sessions).
admin_id	UUID	NULL ON DELETE CASCADE	FK → admin_user IDX	Admin principal (NULL for member sessions).
refresh_token_hash	BYTEA	NOT NULL UNIQUE	UQ	SHA-256 of the current refresh token; rotated on every refresh.
mfa_verified	BOOLEAN	NOT NULL DEFAULT false		True once TOTP passed; admin sessions must be MFA-verified (FR-72).

Field name	Data type	Constraints	Key	Description
device_label	VARCHAR(80)	NULL		Platform and app version, for 'signed-in devices'.
last_activity_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Updated on each request; admin idle > 30 min ⇒ expired.
expires_at	TIMESTAMPTZ	NOT NULL	IDX	Absolute expiry (30 days member, 12 h admin).
revoked_at	TIMESTAMPTZ	NULL		Logout, rotation reuse or Super Admin session reset (FR-74).
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

Table check: `(member_id IS NULL) <> (admin_id IS NULL)`
Table check: `admin_id IS NULL OR mfa_verified`

samaj.device_token Device Token

Retention · Until invalid or logout | Sensitivity · Medium | Volume · ~15k → 70k rows

An FCM registration token for a member's device (FR-53). Marked invalid when FCM reports UNREGISTERED (FR-55).

Field name	Data type	Constraints	Key	Description
device_token_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
member_id	UUID	NOT NULL ON DELETE CASCADE	FK → member ID X	Owner of the device.
platform	VARCHAR(8)	NOT NULL CHECK (platform IN ('ANDROID','IOS'))		ANDROID or IOS.
fcm_token	VARCHAR(512)	NOT NULL UNIQUE	UQ	FCM registration token (APNs via FCM for iOS).
app_version	VARCHAR(20)	NULL		React Native app build that registered the token.
is_valid	BOOLEAN	NOT NULL DEFAULT true		False after UNREGISTERED or logout; no further sends.
last_seen_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last app launch that refreshed this token.
invalidated_at	TIMESTAMPTZ	NULL	IDX	When the token was marked invalid.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

samaj.matrimony_profile Matrimony Profile

Retention · As Member | Sensitivity · Medium | Volume · ~1.5k → 7.5k rows (ASM)

The public summary of a member's matrimony listing (FR-08, FR-20). Contains no biodata — that lives in the separately protected biodata table.

Field name	Data type	Constraints	Key	Description
profile_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
member_id	UUID	NOT NULL UNIQUE ON DELETE RESTRICT	FK → member UQ	Candidate the profile describes; one profile per member.
location_city_id	UUID	NOT NULL ON DELETE RESTRICT	FK → city IDX	Current city for the location filter (may differ from home city).

Field name	Data type	Constraints	Key	Description
managed_by	VARCHAR(10)	NOT NULL DEFAULT 'SELF' CHECK (managed_by IN ('SELF','PARENT','SIBLING','RELATIVE'))		Who maintains the profile (FR-21).
education_level	VARCHAR(20)	NOT NULL CHECK (education_level IN ('SCHOOL','DIPLOMA','GRADUATE','POSTGRADUATE','DOCTORATE','OTHER'))		Filterable education band.
education_detail	VARCHAR(120)	NULL		Free-text degree / institution shown on the card.
occupation	VARCHAR(80)	NULL		Occupation shown in the public summary.
family_background	VARCHAR(20)	NOT NULL		Family background category used by the filter (values per bylaws, OQ-12).
about	VARCHAR(500)	NULL		Short public introduction (moderated).
status	VARCHAR(20)	NOT NULL DEFAULT 'DRAFT' CHECK (status IN ('DRAFT','PENDING_MODERATION','APPROVED','REJECTED','ARCHIVED'))	IDX	Only APPROVED profiles enter search (FR-22, FR-27).
moderation_reason	VARCHAR(500)	NULL		Rejection reason shown to the member (FR-23).
moderated_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	City Admin who moderated.
moderated_at	TIMESTAMPTZ	NULL		Moderation time.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	IDX	Row creation time (UTC).
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

Table check: `status <> 'REJECTED' OR moderation_reason IS NOT NULL`

samaj.biodata Biodata

Retention · As Member | Sensitivity · High | Volume · 1 row per matrimony profile

The private biodata of a matrimony profile, stored as one AES-256-GCM encrypted JSON document. Readable only through samaj.get_biodata(), which enforces the accepted-pair rule.

Field name	Data type	Constraints	Key	Description
profile_id	UUID	NOT NULL ON DELETE CASCADE	PK FK → matrimony_profile	Owning profile (1 : 1).
payload_enc	BYTEA	NOT NULL	PII AES-256-GCM	Encrypted JSON of all biodata fields (family, horoscope details, contacts, etc.). Field set versioned by schema_version (OQ-12).
key_id	VARCHAR(64)	NOT NULL		Identifier of the KMS-wrapped data key used, for rotation.
schema_version	SMALLINT	NOT NULL DEFAULT 1		Version of the biodata field set inside the payload.

Field name	Data type	Constraints	Key	Description
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

No application role has SELECT, INSERT or UPDATE on this table. Writes go through samaj.upsert_biodata() and reads through samaj.get_biodata(); both are SECURITY DEFINER functions. Ciphertext is returned only when the caller and owner form an ACCEPTED biodata_request pair; the API then decrypts it.

samaj.matrimony_photo Matrimony Photo

Retention · As Member | Sensitivity · Medium | Volume · ~5k → 35k rows

A photo on a matrimony profile, moderated against community guidelines before it is shown (FR-09, FR-23). Maximum 5 per profile.

Field name	Data type	Constraints	Key	Description
photo_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
profile_id	UUID	NOT NULL ON DELETE CASCADE	FK → matrimony_profile	Profile the photo belongs to.
storage_key	VARCHAR(255)	NOT NULL		Private object key; served by pre-signed URL.
mime_type	VARCHAR(20)	NOT NULL CHECK (mime_type IN ('image/jpeg','image/png'))		JPG or PNG only (FR-24).
size_bytes	INTEGER	NOT NULL CHECK (size_bytes BETWEEN 1 AND 2097152)		≤ 2 MB.
sort_order	SMALLINT	NOT NULL CHECK (sort_order BETWEEN 1 AND 5)		Display position 1–5; with the unique key this caps photos at 5.
status	VARCHAR(20)	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING','APPROVED','REJECTED'))		Only APPROVED photos are visible to other members.
rejection_reason	VARCHAR(500)	NULL		Reason shown to the member.
moderated_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	Moderating City Admin.
moderated_at	TIMESTAMPTZ	NULL		Moderation time.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	IDX	Row creation time (UTC).

Composite unique: `(profile_id, sort_order)`

Table check: `status <> 'REJECTED' OR rejection_reason IS NOT NULL`

The unique key (profile_id, sort_order) already indexes profile_id for the FK.

samaj.biodata_request Biodata Request

Retention · As Member | Sensitivity · High | Volume · ~20k → 100k rows/year (ASM)

User A's request to exchange biodata with User B (SOP §6.2). ACCEPTED grants mutual access to that pair only. At most one PENDING or ACCEPTED request per pair, in either direction (FR-30).

Field name	Data type	Constraints	Key	Description
request_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).

Field name	Data type	Constraints	Key	Description
requester_profile_id	UUID	NOT NULL ON DELETE CASCADE	FK → matrimony_profile IDX	User A, who sent the request.
recipient_profile_id	UUID	NOT NULL ON DELETE CASCADE	FK → matrimony_profile IDX	User B, who accepts or declines.
pair_low	UUID	NOT NULL GENERATED	IDX	LEAST(requester, recipient); makes the pair direction-independent.
pair_high	UUID	NOT NULL GENERATED	IDX	GREATEST(requester, recipient).
status	VARCHAR(20)	NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING','ACCEPTED','DECLINED','WITHDRAWN'))	IDX	Only ACCEPTED grants biodata access (FR-33, FR-34).
responded_at	TIMESTAMPTZ	NULL		When User B accepted / declined.
resend_allowed_at	TIMESTAMPTZ	NULL	IDX	responded_at + 30 days when DECLINED (FR-39, ASM-08).
withdrawn_at	TIMESTAMPTZ	NULL		When an accepted exchange was revoked (FR-41).
withdrawn_by_profile_id	UUID	NULL ON DELETE SET NULL	FK → matrimony_profile IDX	Which party withdrew.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

Table check: `requester_profile_id <> recipient_profile_id`
Table check: `status = 'PENDING' OR responded_at IS NOT NULL`
Table check: `status <> 'WITHDRAWN' OR (withdrawn_at IS NOT NULL AND withdrawn_by_profile_id IS NOT NULL)`

samaj.content_item Content Item

Retention · 24 months after expiry | Sensitivity · Low | Volume · ~3k rows/year

A news item or ad campaign scheduled in Content Management (SOP §6.3). Goes live at start_at and expires at end_at automatically (FR-47).

Field name	Data type	Constraints	Key	Description
content_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
content_type	VARCHAR(4)	NOT NULL CHECK (content_type IN ('NEWS','AD'))		NEWS or AD.
city_id	UUID	NULL ON DELETE RESTRICT	FK → city IDX	Target city; NULL = all cities (Super Admin only, FR-48).
title	VARCHAR(200)	NOT NULL		Headline / campaign name.
body	TEXT	NULL		News body; required for NEWS.
media_key	VARCHAR(255)	NULL		Image / creative object key (JPG/PNG ≤ 2 MB).
creative_format	VARCHAR(10)	NULL CHECK (creative_format IN ('BANNER','FULLSCREEN'))		Ad creative format.

Field name	Data type	Constraints	Key	Description
media_width	SMALLINT	NULL		Creative width in px; 1080 for full-screen.
media_height	SMALLINT	NULL		Creative height in px; 1920 for full-screen.
start_at	TIMESTAMPTZ	NOT NULL	IDX	Visibility start (UTC).
end_at	TIMESTAMPTZ	NOT NULL	IDX	Visibility end (UTC); must be after start_at (FR-46).
status	VARCHAR(20)	NOT NULL DEFAULT 'DRAFT' CHECK (status IN ('DRAFT','SCHEDULED','LIVE','EXPIRED','CANCELLED'))	IDX	Lifecycle driven by the scheduler.
created_by_admin_id	UUID	NOT NULL ON DELETE RESTRICT	FK → admin_user IDX	Admin who created it.
published_at	TIMESTAMPTZ	NULL		When the scheduler set it LIVE.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

Table check: `end_at > start_at`
Table check: `content_type = 'NEWS' OR (creative_format IS NOT NULL AND media_key IS NOT NULL)`
Table check: `creative_format IS DISTINCT FROM 'FULLSCREEN' OR (media_width = 1080 AND media_height = 1920)`
Table check: `content_type = 'AD' OR body IS NOT NULL`

samaj.layout_setting Layout Setting

Retention · Life of system | Sensitivity · Low | Volume · < 50 rows

Key/value configuration for ad dimensions and feed placement (FR-50, FR-51), e.g. ad_every_n_items = 5.

Field name	Data type	Constraints	Key	Description
setting_key	VARCHAR(60)	NOT NULL	PK	Setting name (e.g. ad_every_n_items, banner_size).
value	JSONB	NOT NULL		Setting value; schema-validated by the API.
updated_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user	Super Admin who last changed it.
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

samaj.panchang_day Panchang

Retention · Indefinite | Sensitivity · Low | Volume · 365 rows/year per location

The effective Tithi, Nakshatra and Yoga for one date and reference location (SOP §6.4). A manual entry overrides API values (FR-62).

Field name	Data type	Constraints	Key	Description
panchang_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
panchang_date	DATE	NOT NULL		Calendar date in IST.

Field name	Data type	Constraints	Key	Description
location_code	VARCHAR(20)	NOT NULL DEFAULT 'DEFAULT'		Reference location for sunrise-based values (ASM-05: one per deployment; OQ-04).
tithi	VARCHAR(40)	NOT NULL		Lunar day.
nakshatra	VARCHAR(40)	NOT NULL		Lunar mansion.
yoga	VARCHAR(40)	NOT NULL		Sun–Moon yoga.
source	VARCHAR(6)	NOT NULL CHECK (source IN ('API','MANUAL'))		API or MANUAL; MANUAL takes precedence.
verified_against_calendar	BOOLEAN	NOT NULL DEFAULT false		Admin confirmed the values against the Samaj Panchang (FR-63).
edited_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	Admin who entered manual values.
fetched_at	TIMESTAMPTZ	NULL		When API values were stored.
published_at	TIMESTAMPTZ	NULL		When the row became valid; on-time KPI compares this with 05:00 IST.
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Last modification time (UTC); maintained by trigger trg_set_updated_at.

Composite unique: `(location_code, panchang_date)`
Table check: `source = 'API' OR (edited_by_admin_id IS NOT NULL AND verified_against_calendar)`
The unique key (location_code, panchang_date) serves the Home-screen lookup and the 04:00 IST check.

samaj.panchang_revision Panchang Revision

Retention · Indefinite | Sensitivity · Low | Volume · ~400–800 rows/year

Append-only history of every API fetch and manual edit of a Panchang day: the edit history shown in the dashboard (FR-62).

Field name	Data type	Constraints	Key	Description
revision_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
panchang_id	UUID	NOT NULL ON DELETE RESTRICT	FK → panchang_day IDX	Panchang day revised.
tithi	VARCHAR(40)	NOT NULL		Tithi in this revision.
nakshatra	VARCHAR(40)	NOT NULL		Nakshatra in this revision.
yoga	VARCHAR(40)	NOT NULL		Yoga in this revision.
source	VARCHAR(6)	NOT NULL CHECK (source IN ('API','MANUAL'))		API or MANUAL.
edited_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	Editor for MANUAL revisions.
api_status	VARCHAR(20)	NULL		API outcome for API revisions (OK, TIMEOUT, INVALID) – ERR_API_PN diagnostics.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

samaj.notification_outbox Notification

Retention · 90 days | Sensitivity · Medium | Volume · ~20k–60k rows/month | Partitioned · monthly

Transactional outbox for push / SMS / email. Written in the same transaction as the business change, then delivered by the worker (≤ 30 s target, NFR-05).

Field name	Data type	Constraints	Key	Description
notification_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	PK IDX	Enqueue time; partition key.
kind	VARCHAR(30)	NOT NULL CHECK (kind IN ('ACCOUNT_DECISION','NEWS_LIVE','BIODATA_REQUEST','BIODATA_RESPONSE','FAMILY_DECISION','PROFILE_DECISION','PANCHANG_ALERT','ARCHIVE_NOTICE','ESCALATION','IMPORT_REPORT'))		Event type.
channel	VARCHAR(6)	NOT NULL CHECK (channel IN ('PUSH','SMS','EMAIL'))		Delivery channel; SMS is the fallback when no valid token (FR-14).
audience	VARCHAR(10)	NOT NULL CHECK (audience IN ('MEMBER','CITY','ALL','ADMIN'))		MEMBER (single), CITY or ALL (news fan-out), ADMIN.
member_id	UUID	NULL ON DELETE CASCADE	FK → member IDX	Recipient when audience = MEMBER.
city_id	UUID	NULL ON DELETE RESTRICT	FK → city IDX	Target city when audience = CITY.
admin_id	UUID	NULL ON DELETE RESTRICT	FK → admin_user IDX	Recipient when audience = ADMIN.
content_id	UUID	NULL ON DELETE SET NULL	FK → content_item IDX	News item that triggered a NEWS_LIVE fan-out.
payload	JSONB	NOT NULL DEFAULT '{}'::jsonb		Template id + non-PII parameters. Never contains biodata, phone or address.
state	VARCHAR(10)	NOT NULL DEFAULT 'QUEUED' CHECK (state IN ('QUEUED','SENDING','SENT','PARTIAL','FAILED'))	IDX	Delivery state.
attempts	SMALLINT	NOT NULL DEFAULT 0		Delivery attempts (max 5 with back-off).
sent_at	TIMESTAMPTZ	NULL		When the provider accepted the send.
success_count	INTEGER	NULL		Devices / recipients delivered (fan-out).
failure_count	INTEGER	NULL		Devices / recipients failed.

Table check: `member_id IS NOT NULL OR audience <> 'MEMBER'`

samaj.import_job Import Job

Retention · 12 months | Sensitivity · Medium | Volume · ~500 rows/year

A CSV/XLSX member-list upload (SOP §6.5). Validated → dry-run report → committed. A header mismatch rejects the file before any write (FR-76).

Field name	Data type	Constraints	Key	Description
import_job_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
city_id	UUID	NOT NULL ON DELETE RESTRICT	FK → city IDX	City the list belongs to (City Admin scope).
uploaded_by_admin_id	UUID	NOT NULL ON DELETE RESTRICT	FK → admin_user IDX	Uploading admin.
file_key	VARCHAR(255)	NOT NULL		Encrypted object key of the uploaded file.
file_format	VARCHAR(4)	NOT NULL CHECK (file_format IN ('CSV','XLSX'))		CSV or XLSX (FR-75).
template_version	VARCHAR(20)	NOT NULL		Header template the file was validated against.
status	VARCHAR(20)	NOT NULL DEFAULT 'VALIDATING' CHECK (status IN ('VALIDATING','REJECTED','DRY_RUN_READY','COMMITTING','COMMITTED','FAILED'))		Import lifecycle.
total_rows	INTEGER	NULL CHECK (total_rows <= 50000)		Data rows in file (≤ 50,000).
new_count	INTEGER	NOT NULL DEFAULT 0		Rows that create new members.
updated_count	INTEGER	NOT NULL DEFAULT 0		Rows matched on mobile and updated.
conflict_count	INTEGER	NOT NULL DEFAULT 0		Mobile matches with differing name + DOB (FR-78).
error_count	INTEGER	NOT NULL DEFAULT 0		Invalid rows.
report_key	VARCHAR(255)	NULL		Object key of the downloadable row-level report.
committed_at	TIMESTAMPTZ	NULL		When the commit finished.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

samaj.import_row Import Row

Retention · Payload 30 days; row 12 months | Sensitivity · High | Volume · ~10k rows per import

One validated row of an import, with its dedup outcome (NEW / UPDATE / CONFLICT / ERROR). Staged data is encrypted and wiped 30 days after commit.

Field name	Data type	Constraints	Key	Description
import_job_id	UUID	NOT NULL ON DELETE CASCADE	PK FK → import_job IDX	Parent import.
row_number	INTEGER	NOT NULL	PK	1-based row number in the file.

Field name	Data type	Constraints	Key	Description
outcome	VARCHAR(8)	NOT NULL CHECK (outcome IN ('NEW','UPDATE','CONFLICT','ERROR'))		Dedup / validation result.
mobile_hash	BYTEA	NULL		HMAC of the row's mobile, used for matching.
matched_member_id	UUID	NULL ON DELETE SET NULL	FK → member IDX	Existing member matched (UPDATE / CONFLICT) or created (NEW).
payload_enc	BYTEA	NULL		PII AES-256-GCM Encrypted staged row values for conflict review.
error_message	VARCHAR(500)	NULL		Row-level validation message (no PII).
resolution	VARCHAR(10)	NULL CHECK (resolution IN ('APPLIED','SKIPPED'))		Admin decision on a conflict.

samaj.export_job Export Job

Retention · 24 months | Sensitivity · Medium | Volume · ~1k rows/year

Record of an 'Export Filtered' (FR-82). The password and the file are never stored; only who exported what, and how many rows.

Field name	Data type	Constraints	Key	Description
export_job_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
city_id	UUID	NULL ON DELETE RESTRICT	FK → city IDX	City scope of the export (NULL = all, Super Admin).
exported_by_admin_id	UUID	NOT NULL ON DELETE RESTRICT	FK → admin_user IDX	Admin who exported.
filters	JSONB	NOT NULL		Filter criteria applied (age band, area, city).
columns_exported	TEXT[]	NOT NULL		Column list; validated against an allow-list that excludes biodata (FR-84).
row_count	INTEGER	NOT NULL CHECK (row_count >= 0)		Rows written to the encrypted file.
file_format	VARCHAR(4)	NOT NULL CHECK (file_format IN ('CSV','XLSX'))		CSV or XLSX.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		Row creation time (UTC).

samaj.audit_log Audit Log

Retention · ≥ 24 months | Sensitivity · High | Volume · ~20k–50k rows/month | Partitioned · monthly

Append-only record of every admin action and data-principal event (FR-86, FR-87). No FKs, so log rows outlive the objects they describe.

Field name	Data type	Constraints	Key	Description
audit_id	BIGINT	NOT NULL DEFAULT nextval('samaj.audit_log_audit_id_seq')	PK	Monotonic identifier (identity).

Field name	Data type	Constraints	Key	Description
occurred_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	PK IDX	Event time (UTC); partition key.
actor_type	VARCHAR(8)	NOT NULL CHECK (actor_type IN ('ADMIN','MEMBER','SYSTEM'))		Who acted.
actor_id	UUID	NULL	IDX	admin_id or member_id of the actor; NULL for SYSTEM.
action	VARCHAR(40)	NOT NULL	IDX	e.g. MEMBER_APPROVE, MEMBER_REJECT, PHOTO_REJECT, IMPORT_COMMIT, EXPORT, PANCHANG_OVERRIDE, ROLE_ASSIGN, MEMBER_ERASED.
target_table	VARCHAR(40)	NULL	IDX	Table of the affected row.
target_id	VARCHAR(64)	NULL	IDX	Primary key of the affected row.
city_id	UUID	NULL	IDX	City context, for the city filter in the audit view (FR-88).
before_state	JSONB	NULL		Changed fields before (PII masked; biodata never included).
after_state	JSONB	NULL		Changed fields after (PII masked; biodata never included).
source_ip	INET	NULL		Client IP of the request.
request_id	UUID	NULL		Correlation id from the API, linking to application logs.

samaj.security_event Security Event

Retention · ≥ 24 months | Sensitivity · High | Volume · ~1k–5k rows/month | Partitioned · monthly

Denied or suspicious access, above all every denied biodata read (FR-37), plus OTP lockouts, MFA failures (ERR_AUTH_01) and cross-city 403s.

Field name	Data type	Constraints	Key	Description
event_id	BIGINT	NOT NULL DEFAULT nextval('samaj.security_event_event_id_seq')	PK	Monotonic identifier.
occurred_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	PK IDX	Event time; partition key.
event_type	VARCHAR(30)	NOT NULL CHECK (event_type IN ('BIODATA_DENIED','CROSS_CITY_DENIED','ID_DOC_DENIED','OTP_LOCKED','OTP_RATE_LIMIT','MFA_FAILED','TOKEN_REUSE'))	IDX	Category.
actor_type	VARCHAR(8)	NOT NULL CHECK (actor_type IN ('ADMIN','MEMBER','ANONYMOUS'))		Caller type.
actor_id	UUID	NULL	IDX	Caller's admin_id / member_id if authenticated.
endpoint	VARCHAR(120)	NOT NULL		API route that was called (e.g. GET /api/v1/biodata/{id}).
target_id	VARCHAR(64)	NULL		Object that was requested (e.g. profile_id).

Field name	Data type	Constraints	Key	Description
reason	VARCHAR(200)	NULL		Why access was denied (no PII).
source_ip	INET	NULL		Client IP.

samaj.feedback_report Feedback / Report

Retention · 24 months | Sensitivity · Medium | Volume · ~2k rows/year

In-app feedback, a profile report (FR-91) or a DPDP grievance (FR-100), reviewed daily in the dashboard.

Field name	Data type	Constraints	Key	Description
feedback_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
member_id	UUID	NULL ON DELETE SET NULL	FK → member IDX	Member who submitted it; NULL after the reporter is erased.
category	VARCHAR(16)	NOT NULL CHECK (category IN ('FEEDBACK','PROFILE_REPORT','GRIEVANCE'))	IDX	Type of submission.
target_profile_id	UUID	NULL ON DELETE SET NULL	FK → matrimony_profile IDX	Reported matrimony profile.
message	VARCHAR(2000)	NOT NULL	PII	Submission text.
status	VARCHAR(20)	NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN','IN_REVIEW','RESOLVED','DISMISSED'))		Handling state.
handled_by_admin_id	UUID	NULL ON DELETE SET NULL	FK → admin_user IDX	Admin handling it.
resolved_at	TIMESTAMPTZ	NULL		Resolution time.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	IDX	Row creation time (UTC).

Table check: `category <> 'PROFILE_REPORT' OR target_profile_id IS NOT NULL`

samaj.data_rights_request Data Rights Request

Retention · 24 months after completion | Sensitivity · Medium | Volume · ~100s rows/year

A DPDP data-principal request: access copy, erasure (OTP-confirmed) or consent withdrawal, which triggers erasure. Completed within 30 days (FR-98, FR-99).

Field name	Data type	Constraints	Key	Description
rights_request_id	UUID	NOT NULL DEFAULT gen_random_uuid()	PK	Surrogate key (UUID v4).
member_id	UUID	NOT NULL ON DELETE RESTRICT	FK → member IDX	Requesting data principal (row is anonymised, not deleted, after erasure).
request_type	VARCHAR(20)	NOT NULL CHECK (request_type IN ('ACCESS','ERASURE','CONSENT_WITHDRAWAL','RETENTION_PURGE'))		Kind of request.

Field name	Data type	Constraints	Key	Description
status	VARCHAR(20)	NOT NULL DEFAULT 'RECEIVED' CHECK (status IN ('RECEIVED','OTP_CONFIRMED','IN_PROGRESS','COMPLETED','REJECTED'))		Workflow state.
otp_confirmed_at	TIMESTAMPTZ	NULL		OTP confirmation for erasure (FR-98).
requested_at	TIMESTAMPTZ	NOT NULL DEFAULT now()		When the request was raised.
due_at	TIMESTAMPTZ	NOT NULL DEFAULT now() + INTERVAL '30 days'	**IDX**	requested_at + 30 days (statutory target); set at insert, never updated.
completed_at	TIMESTAMPTZ	NULL		When the request was fulfilled.
handled_by_admin_id	UUID	NULL ON DELETE SET NULL	**FK** → admin_user **IDX**	Super Admin / grievance officer handling it.

Table check: `status <> 'COMPLETED' OR completed_at IS NOT NULL`

5 Performance Optimization & Storage Strategy

Indexing Strategy

Every foreign key is indexed. Partial indexes cover the dominant "only the live ones" lookups: approval queues, live content, valid device tokens, open requests. Trigram GIN indexes serve directory name search, and BRIN indexes cover time-range scans on append-only logs. The monthly maintenance in SOP §11 runs `REINDEX CONCURRENTLY` on indexes whose bloat estimate exceeds 30%.

Index	Table	Columns	Type	Serves query / purpose
ux_admin_user_one_city_admin	**admin_user**	(city_id WHERE role = 'CITY_ADMIN' AND status = 'ACTIVE')	UNIQUE B-tree partial	Enforces exactly one active City Admin per city (FR-70)
ix_household_city	**household**	(city_id)	B-tree	FK index; City Admin household lists
ix_household_family_name_trgm	**household**	(family_name gin_trgm_ops)	GIN	Partial/fuzzy family-name search in the directory
ix_member_city_status	**member**	(city_id, status, created_at)	B-tree composite	City Admin approval queue, oldest first (FR-11); 72 h escalation scan
ix_member_household	**member**	(household_id)	B-tree	FK index; household view (FR-67)
ux_member_one_head	**member**	(household_id WHERE is_household_head)	UNIQUE B-tree partial	Exactly one head per household
ix_member_name_trgm	**member**	(full_name gin_trgm_ops WHERE status = 'ACTIVE' AND deleted_at IS NULL)	GIN partial	Directory name search over active members (FR-65)
ix_member_last_login	**member**	(last_login_at WHERE status IN ('ACTIVE','ARCHIVED') AND erased_at IS NULL)	B-tree partial	Nightly archive / purge candidate scan (FR-92–94)
ix_member_reviewed_by	**member**	(reviewed_by_admin_id)	B-tree	FK index
ix_member_deleted	**member**	(deleted_at WHERE deleted_at IS NOT NULL)	B-tree partial	Archive Recovery list and 30-day hard-purge scan
ix_family_member_household	**family_member**	(household_id WHERE status = 'APPROVED')	B-tree partial	Household view lists approved relatives only
ix_family_member_pending	**family_member**	(status, created_at WHERE status = 'PENDING')	B-tree composite partial	Family Additions approval queue
ix_family_member_requested_by	**family_member**	(requested_by_member_id)	B-tree	FK index; member's own requests
ix_family_member_linked	**family_member**	(linked_member_id)	B-tree	FK index
ix_family_member_reviewed_by	**family_member**	(reviewed_by_admin_id)	B-tree	FK index
ix_identity_document_member	**identity_document**	(member_id)	B-tree	FK index; approval screen preview
ix_identity_document_reviewed_by	**identity_document**	(reviewed_by_admin_id)	B-tree	FK index
ix_identity_document_purge	**identity_document**	(purge_after WHERE purge_after IS NOT NULL)	B-tree partial	Daily purge job
ix_consent_member	**consent**	(member_id)	B-tree	FK index; consent check at login

Index	Table	Columns	Type	Serves query / purpose
ix_otp_challenge_mobile_time	otp_challenge	(mobile_hash, created_at DESC)	B-tree composite	Latest OTP lookup and rolling-hour send count (FR-04)
ix_otp_challenge_created	otp_challenge	(created_at)	BRIN	Hourly purge
ix_auth_session_member	auth_session	(member_id WHERE revoked_at IS NULL)	B-tree partial	Revoke all sessions of a member
ix_auth_session_admin	auth_session	(admin_id WHERE revoked_at IS NULL)	B-tree partial	Session reset (ERR_AUTH_01)
ix_auth_session_expires	auth_session	(expires_at)	B-tree	Daily purge
ix_device_token_member	device_token	(member_id WHERE is_valid)	B-tree partial	Push fan-out to a member's live devices
ix_device_token_invalidated	device_token	(invalidated_at WHERE NOT is_valid)	B-tree partial	Purge job
ix_matrimony_profile_moderation	matrimony_profile	(status, created_at WHERE status = 'PENDING_MODERATION')	B-tree composite partial	Photo moderation queue
ix_matrimony_profile_location	matrimony_profile	(location_city_id)	B-tree	FK index
ix_matrimony_profile_moderated_by	matrimony_profile	(moderated_by_admin_id)	B-tree	FK index
ix_matrimony_photo_pending	matrimony_photo	(created_at WHERE status = 'PENDING')	B-tree partial	Moderation queue
ix_matrimony_photo_moderated_by	matrimony_photo	(moderated_by_admin_id)	B-tree	FK index
ux_biodata_request_open_pair	biodata_request	(pair_low, pair_high WHERE status IN ('PENDING','ACCEPTED'))	UNIQUE B-tree composite partial	One open request per pair in either direction (FR-30); also the accepted-pair lookup used by get_biodata()
ix_biodata_request_recipient	biodata_request	(recipient_profile_id, status, created_at DESC)	B-tree composite	Received tab (FR-40)
ix_biodata_request_requester	biodata_request	(requester_profile_id, created_at DESC)	B-tree composite	Sent tab; daily cap count of 20/day (FR-42)
ix_biodata_request_declined_pair	biodata_request	(pair_low, pair_high, resend_allowed_at WHERE status = 'DECLINED')	B-tree composite partial	30-day resend lock check (FR-39)
ix_biodata_request_withdrawn_by	biodata_request	(withdrawn_by_profile_id)	B-tree	FK index
ix_content_item_feed	content_item	(status, city_id, start_at DESC WHERE status = 'LIVE')	B-tree composite partial	Member feed: live items for a city + all-city, newest first (FR-49)
ix_content_item_due	content_item	(start_at WHERE status = 'SCHEDULED')	B-tree partial	Scheduler: items due to go live
ix_content_item_expiring	content_item	(end_at WHERE status = 'LIVE')	B-tree partial	Scheduler: items due to expire
ix_content_item_city	content_item	(city_id)	B-tree	FK index; calendar view per city
ix_content_item_created_by	content_item	(created_by_admin_id)	B-tree	FK index
ix_panchang_day_edited_by	panchang_day	(edited_by_admin_id)	B-tree	FK index

Index	Table	Columns	Type	Serves query / purpose
ix_panchang_revision_day	panchang_revision	(panchang_id, created_at DESC)	B-tree composite	Edit history panel
ix_panchang_revision_edited_by	panchang_revision	(edited_by_admin_id)	B-tree	FK index
ix_notification_outbox_queue	notification_outbox	(state, created_at WHERE state IN ('QUEUED','SENDING'))	B-tree composite partial	Worker poll with FOR UPDATE SKIP LOCKED
ix_notification_outbox_member	notification_outbox	(member_id)	B-tree	FK index; member notification history
ix_notification_outbox_content	notification_outbox	(content_id)	B-tree	FK index; push-latency report per news item
ix_notification_outbox_city	notification_outbox	(city_id)	B-tree	FK index
ix_notification_outbox_admin	notification_outbox	(admin_id)	B-tree	FK index
ix_import_job_city_time	import_job	(city_id, created_at DESC)	B-tree composite	Import history per city
ix_import_job_uploaded_by	import_job	(uploaded_by_admin_id)	B-tree	FK index
ix_import_row_conflicts	import_row	(import_job_id WHERE outcome = 'CONFLICT' AND resolution IS NULL)	B-tree partial	Conflict review list
ix_import_row_member	import_row	(matched_member_id)	B-tree	FK index
ix_export_job_admin_time	export_job	(exported_by_admin_id, created_at DESC)	B-tree composite	FK index; monthly audit review
ix_export_job_city	export_job	(city_id)	B-tree	FK index
ix_audit_log_occurred	audit_log	(occurred_at)	BRIN	Date-range filter in the monthly audit view
ix_audit_log_actor	audit_log	(actor_id, occurred_at DESC)	B-tree composite	Filter by admin (FR-88)
ix_audit_log_city_action	audit_log	(city_id, action, occurred_at DESC)	B-tree composite	Filter by city and action
ix_audit_log_target	audit_log	(target_table, target_id)	B-tree composite	History of one record
ix_security_event_occurred	security_event	(occurred_at)	BRIN	Daily log review
ix_security_event_type	security_event	(event_type, occurred_at DESC)	B-tree composite	Biodata-denial report; alerting
ix_security_event_actor	security_event	(actor_id, occurred_at DESC)	B-tree composite	Investigate a suspicious account
ix_feedback_report_open	feedback_report	(category, created_at WHERE status IN ('OPEN','IN_REVIEW'))	B-tree composite partial	Daily review queue
ix_feedback_report_member	feedback_report	(member_id)	B-tree	FK index
ix_feedback_report_target	feedback_report	(target_profile_id)	B-tree	FK index; reports against a profile
ix_feedback_report_handler	feedback_report	(handled_by_admin_id)	B-tree	FK index
ix_data_rights_request_due	data_rights_request	(due_at WHERE status NOT IN ('COMPLETED','REJECTED'))	B-tree partial	Overdue / SLA tracking; nightly erasure job
ix_data_rights_request_member	data_rights_request	(member_id)	B-tree	FK index

Index	Table	Columns	Type	Serves query / purpose
ix_data_rights_request_handler	data_rights_request	(handled_by_admin_id)	B-tree	FK index

Primary keys, UNIQUE columns and composite unique constraints also create implicit unique B-tree indexes (see the data dictionary); they are not repeated here.

Critical Queries

Query	Index used	Target (P95, DB time)
Login / duplicate check by mobile (FR-02, FR-05)	member.mobile_hash (UQ)	< 5 ms
OTP send count, rolling hour (FR-04)	ix_otp_challenge_mobile_time	< 5 ms
City Admin approval queue, oldest first (FR-11)	ix_member_city_status	< 20 ms
Matrimony search, 4 filters (FR-25)	ix_matrimony_search_mv_filters	< 150 ms at 7,500 approved profiles (well within NFR-04)
Biodata read, pair check (FR-35)	ux_biodata_request_open_pair + biodata PK	< 5 ms
Received / Sent request tabs (FR-40)	ix_biodata_request_recipient / _requester	< 10 ms
News feed for a city + all-city (FR-49)	ix_content_item_feed	< 10 ms
Directory name search (FR-65)	ix_member_name_trgm	< 50 ms
Today's Panchang (FR-59)	uq_panchang_day_location_code_panchang_date	< 2 ms
Outbox poll (NFR-05 ≤ 30 s push)	ix_notification_outbox_queue	< 5 ms, polled every 1 s
Monthly audit view, filtered (FR-88)	ix_audit_log_city_action / BRIN	< 300 ms on the replica

Matrimony Search

Search reads the materialized view, never `biodata`. The scheduler refreshes it concurrently, so readers are not blocked. The re-join on `matrimony_profile` means a profile that has been rejected, archived or erased disappears at once, even before the next refresh.

```
-- Called by the scheduler every 4 hours (SOP §6.2), and immediately after any erasure or purge.
CREATE FUNCTION samaj.refresh_matrimony_search() RETURNS void
LANGUAGE sql SECURITY DEFINER SET search_path = samaj, pg_temp AS
$$ REFRESH MATERIALIZED VIEW CONCURRENTLY samaj.matrimony_search_mv $$;
GRANT EXECUTE ON FUNCTION samaj.refresh_matrimony_search() TO samaj_worker, samaj_jobs;

SELECT s.profile_id, s.age_years, s.education_level, s.education_detail,
       s.occupation, s.location_city_id, s.cover_photo_key
  FROM samaj.matrimony_search_mv s
  JOIN samaj.matrimony_profile p               -- drops profiles hidden since the last refresh
    ON p.profile_id = s.profile_id AND p.status = 'APPROVED'
 WHERE s.gender = :gender
   AND s.age_years BETWEEN 24 AND 30
   AND s.education_level = ANY (ARRAY['GRADUATE','POSTGRADUATE'])
   AND s.member_id <> :me                      -- never your own profile (FR-27)
 ORDER BY s.profile_id
 LIMIT 20;                                      -- keyset pagination on profile_id
```

Concurrency

> **BIODATA REQUESTS ARE RACE-FREE**
>
> Two taps on "Request Biodata", or A and B requesting each other at the same moment, must not create two open requests (FR-30). The partial unique index on (`pair_low`, `pair_high`) makes this impossible at any concurrency. Locking the sender's profile row serialises A's own sends, so the 20-per-day cap (FR-42) cannot be beaten by parallel calls.

```
BEGIN;
-- Lock the sender's profile row: concurrent sends by A are serialised, so the daily cap is race-free.
SELECT 1 FROM samaj.matrimony_profile WHERE profile_id = :a FOR UPDATE;

-- FR-42: at most 20 new requests per IST calendar day (API rejects when count >= 20).
SELECT count(*) AS sent_today
  FROM samaj.biodata_request
 WHERE requester_profile_id = :a
   AND created_at >= date_trunc('day', now() AT TIME ZONE 'Asia/Kolkata') AT TIME ZONE 'Asia/Kolkata';

-- FR-39: 30-day lock after a decline, in either direction.
SELECT EXISTS (SELECT 1 FROM samaj.biodata_request
               WHERE pair_low = LEAST(CAST(:a AS uuid), CAST(:b AS uuid)) AND pair_high = GREATEST(CAST(:a AS
uuid), CAST(:b AS uuid))
                 AND status = 'DECLINED' AND resend_allowed_at > now()) AS locked;

-- FR-30: the partial unique index makes a second open request (A→B or B→A) impossible.
INSERT INTO samaj.biodata_request (requester_profile_id, recipient_profile_id)
VALUES (:a, :b)
ON CONFLICT (pair_low, pair_high) WHERE status IN ('PENDING','ACCEPTED') DO NOTHING
RETURNING request_id;                       -- 0 rows → "Request already exists"
COMMIT;
```

Accepting or declining is a single conditional `UPDATE`: a second tap, or a race with a withdrawal, affects 0 rows.

```
-- Accept: a single conditional UPDATE. Only User B, and only while PENDING (FR-32, FR-33).
UPDATE samaj.biodata_request
   SET status = 'ACCEPTED', responded_at = now()
 WHERE request_id = :rid AND recipient_profile_id = :b AND status = 'PENDING'
RETURNING requester_profile_id;             -- 0 rows → already answered, withdrawn or not yours

-- Decline sets the 30-day resend lock in the same statement:
-- SET status = 'DECLINED', responded_at = now(), resend_allowed_at = now() + INTERVAL '30 days'
```

The same pattern protects the other race-prone operations:

- **Registration** uses `INSERT … ON CONFLICT (mobile_hash) DO NOTHING`.
- **Import commit** upserts on `mobile_hash`. Rows where the name and date of birth differ were already classified CONFLICT and are excluded (FR-78).
- **City Admin assignment** relies on the unique violation from `ux_admin_user_one_city_admin`, which the API returns as "City already has an admin" (FR-70).
- **Content go-live** uses `UPDATE … WHERE status = 'SCHEDULED' AND start_at <= now()`, so two scheduler instances cannot publish an item twice.

Partitioning & Sharding

`audit_log`, `security_event` and `notification_outbox` are range-partitioned by month. These are the only tables that grow without bound. Retention then becomes a metadata operation (export, then drop a partition) rather than a large `DELETE`. The monthly audit review prunes to one partition. `pg_partman` pre-creates 3 months ahead. All other tables stay small: under 200,000 rows at the 50,000-member target. Sharding is not needed at this scale; at 50+ cities, `city_id` on every scoped table would allow a later split by city.

```
SELECT partman.create_parent(p_parent_table => 'samaj.audit_log',
                             p_control      => 'occurred_at',
                             p_interval     => '1 month',
                             p_premake      => 3);
UPDATE partman.part_config
   SET retention = '24 months', retention_keep_table = false
 WHERE parent_table = 'samaj.audit_log';
-- Same for samaj.security_event (24 months) and samaj.notification_outbox (retention = '3 months').
```

Data Retention & Archiving

Table	Retention	Clock starts	Archive / cold storage	Purge method	Sensitivity
city	**Life of system**	—	—	Never deleted; is_active = false retires a city	Low
admin_user	**Life of account + 24 months**	deactivated_at	—	Anonymise name/email and wipe secrets 24 months after deactivation; row kept so audit references stay valid	● High
household	**As Member**	Last linked member purged	—	Deleted by retention job when no member or family_member remains	● High
member	**Archive 12 mo inactive; purge 24 mo**	last_login_at (inactivity) · deleted_at (30-day recovery)	status = ARCHIVED at 12 months (hidden from search/directory)	Anonymise in place at 24 months or on erasure: wipe PII, randomise mobile_hash, delete dependants	● High
family_member	**As Member**	Household purge or requester erasure	—	DELETE (CASCADE from household)	● High
identity_document	**90 days after decision**	Member review decision (reviewed_at)	None	Delete object + row 90 days after the approval decision (ASM, OQ-10)	● High
consent	**Life of account + 24 months**	Member erased_at	—	DELETE 24 months after erasure (proof of consent kept until then)	Medium
otp_challenge	**24 hours**	created_at	None	Hourly DELETE of rows older than 24 h	Medium
auth_session	**30 days after end**	expires_at / revoked_at	None	Daily DELETE 30 days after expiry or revocation	Medium
device_token	**Until invalid or logout**	is_valid = false	None	Daily DELETE 30 days after invalidation	Medium
matrimony_profile	**As Member**	Member archive / purge	status = ARCHIVED with the member	DELETE on member purge (cascades to biodata, photos, requests)	Medium
biodata	**As Member**	Profile deletion	None (never archived or exported)	DELETE (CASCADE from matrimony_profile)	● High
matrimony_photo	**As Member**	Profile deletion or photo rejection	None	Rejected: object + row deleted after 30 days; otherwise CASCADE with profile	Medium
biodata_request	**As Member**	Either profile deleted	None	DELETE (CASCADE from matrimony_profile)	● High

Table	Retention	Clock starts	Archive / cold storage	Purge method	Sensitivity
content_item	**24 months after expiry**	end_at	Media moved to infrequent-access storage on expiry	DELETE row + media 24 months after end_at	Low
layout_setting	**Life of system**	—	Changes captured in audit_log	Never	Low
panchang_day	**Indefinite**	—	—	Never (cultural reference data)	Low
panchang_revision	**Indefinite**	—	—	Never	Low
notification_outbox	**90 days**	created_at	None	Drop monthly partitions older than 90 days (pg_partman)	Medium
import_job	**12 months**	created_at	Report file kept in object storage for the same period	DELETE job (CASCADE import_row) + uploaded file after 12 months	Medium
import_row	**Payload 30 days; row 12 months**	import_job.committed_at	None	payload_enc set NULL after 30 days; rows CASCADE with job at 12 months	● High
export_job	**24 months**	created_at	None	DELETE after 24 months	Medium
audit_log	**≥ 24 months**	occurred_at	Monthly partition exported to S3 Object Lock (Parquet) before drop	Drop partitions older than 24 months (never by the member retention job)	● High
security_event	**≥ 24 months**	occurred_at	Exported with audit_log to S3 Object Lock	Drop partitions older than 24 months	● High
feedback_report	**24 months**	created_at	None	DELETE after 24 months; reporter link cleared on erasure	Medium
data_rights_request	**24 months after completion**	completed_at	None	DELETE 24 months after completion (evidence of compliance)	Medium

ERASURE VS. HISTORY

A purge at 24 months, or a DPDP erasure, must not break the 24-month audit trail. The retention job therefore does not delete the member. It runs these steps in one transaction:

1. Deletes the member's matrimony profile. This cascades to the biodata, photos and biodata requests.
2. Deletes the member's ID documents, device tokens, sessions and pending family requests.
3. Anonymises the `member` row: wipes `full_name`, `dob`, `mobile_enc`, `mobile_last4` and `address_enc`, replaces `mobile_hash` with random bytes, and sets `erased_at`.
4. Writes `MEMBER_ERASED` to the audit log and refreshes the search view.

Audit rows refer only to `member_id` and never hold PII, so nothing left can identify the person. Archive Recovery (FR-96) works on soft-deleted rows (`deleted_at`) for 30 days, before anonymisation runs.

6 Security, Compliance & Access Control

Authentication & Authorization

Applications connect to PgBouncer with per-role credentials that AWS Secrets Manager rotates every 30 days. Tasks fetch them at start-up through their IAM role, so no password is kept in code, images or environment files. **Humans** use IAM database authentication over the VPN, with short-lived tokens, after SSO with MFA, into named read-only roles. The RDS master user is break-glass only and never used by an application.

Member and admin authentication (OTP, TOTP, JWT) is handled by the API. The database stores only hashes: Argon2id for passwords and OTP codes, SHA-256 for refresh tokens.

Role-Based Access Control

Database roles mirror the processes, not the product roles. Super Admin, City Admin and Member are application roles, enforced by the API and backed by RLS. Only `samaj_jobs` may delete rows. No role, apart from the owner that runs migrations, holds any privilege on `biodata`.

```
-- Created once per environment by the platform team; passwords are rotated by Secrets Manager.
CREATE ROLE samaj_owner   NOLOGIN;
CREATE ROLE samaj_app     LOGIN;
CREATE ROLE samaj_worker  LOGIN;
CREATE ROLE samaj_jobs    LOGIN;
CREATE ROLE samaj_ops_ro  LOGIN;
CREATE ROLE samaj_auditor LOGIN;
ALTER SCHEMA samaj OWNER TO samaj_owner;
GRANT USAGE ON SCHEMA samaj TO samaj_app, samaj_worker, samaj_jobs, samaj_ops_ro, samaj_auditor;
```

Role	Used by	Purpose & limits
samaj_owner	Alembic migrations (CI/CD, NOLOGIN owner)	Owns every object, the SECURITY DEFINER biodata functions and the search view refresh. Only assumed by the migration pipeline via SET ROLE.
samaj_app	FastAPI service (SQLAlchemy, via PgBouncer)	Day-to-day API reads/writes. No DELETE anywhere and no direct access to biodata (EXECUTE on get_biodata / upsert_biodata only). Insert-only on logs.
samaj_worker	Background workers (scheduler, push, Panchang fetch, import processor)	Publishes/expires content, delivers notifications, fetches Panchang, validates and commits imports.
samaj_jobs	Nightly retention, erasure & purge jobs	The only role that DELETEs. Archives, anonymises and purges per the retention table; partitions are dropped by pg_partman running as samaj_owner.
samaj_ops_ro	Senior technical staff over VPN (SOP §4)	Read-only troubleshooting access. Members only through the masking view; no access to biodata, secrets or import payloads.
samaj_auditor	Super Admin monthly audit export / compliance reviewer	Read-only on the audit trail and compliance records.

Table	samaj_owner	samaj_app	samaj_worker	samaj_jobs	samaj_ops_ro	samaj_auditor
city	all	R	R	—	R	—
admin_user	all	CRU	R	—	—	—
household	all	CRU	CR	RD	R	—
member	all	CRU	CRU	RU	masked	—
family_member	all	CRU	—	RD	—	—
identity_document	all	CR	—	RD	—	—
consent	all	CRU	C	RD	—	R

Table	samaj_owner	samaj_app	samaj_worker	samaj_jobs	samaj_ops_ro	samaj_auditor
otp_challenge	all	CRU	—	D	—	—
auth_session	all	CRU	—	RD	—	—
device_token	all	CRU	RU	RD	—	—
matrimony_profile	all	CRU	R	RUD	R	—
biodata	all	—	—	—	—	—
matrimony_photo	all	CRU	—	RD	—	—
biodata_request	all	CRU	—	—	R	—
content_item	all	CRU	RU	RD	R	—
layout_setting	all	RU	—	—	R	—
panchang_day	all	CRU	CRU	—	R	—
panchang_revision	all	CR	CR	—	R	—
notification_outbox	all	C	CRU	C	R	—
import_job	all	CRU	RU	RD	R	—
import_row	all	RU	CRU	RU	—	—
export_job	all	CR	—	RD	R	R
audit_log	all	C	C	C	—	R
security_event	all	C	—	—	—	R
feedback_report	all	CRU	—	RUD	—	—
data_rights_request	all	CRU	—	RUD	—	R

C = INSERT · R = SELECT · U = UPDATE · D = DELETE · masked = reads through masking view only

Biodata Pair-Access Enforcement

> **PRIVACY RULE IN THE DATABASE (SOP §6.2, §8 · FR-33–37)**
>
> The API can obtain biodata only by calling `samaj.get_biodata()`. The function returns ciphertext only when the session is a member and either owns the profile or forms an **ACCEPTED** pair with it. Admin, system and anonymous sessions always get nothing. Every denial is written to `security_event` inside the database, so a forgotten log call in the API cannot lose it.
>
> This protects against the realistic failure, which is a code path that forgets the check: an admin screen, a search join or an export. Because the API decrypts the payload, a fully compromised API host is outside what the database alone can stop. That case is covered by KMS key policies and the release-gate privacy test suite.

```
-- No application role can touch samaj.biodata directly.
REVOKE ALL ON samaj.biodata FROM PUBLIC, samaj_app, samaj_worker, samaj_jobs, samaj_ops_ro, samaj_auditor;

-- Read path: returns ciphertext only to the owner or to the other member of an ACCEPTED pair.
-- Admin and system sessions always get nothing. Every denial is logged (FR-35, FR-36, FR-37).
CREATE FUNCTION samaj.get_biodata(p_owner_profile_id uuid)
RETURNS TABLE (payload_enc bytea, key_id varchar, schema_version smallint)
LANGUAGE plpgsql SECURITY DEFINER SET search_path = samaj, pg_temp AS $$
DECLARE
  v_role   text := current_setting('app.actor_role', true);
  v_member uuid := nullif(current_setting('app.member_id', true), '')::uuid;
  v_viewer uuid;
BEGIN
  IF v_role = 'MEMBER' AND v_member IS NOT NULL THEN
    SELECT p.profile_id INTO v_viewer FROM matrimony_profile p WHERE p.member_id = v_member;
    RETURN QUERY
      SELECT b.payload_enc, b.key_id, b.schema_version
        FROM biodata b
       WHERE b.profile_id = p_owner_profile_id
```

```
        AND (v_viewer = p_owner_profile_id
             OR EXISTS (SELECT 1 FROM biodata_request r
                        WHERE r.status = 'ACCEPTED'
                          AND r.pair_low  = LEAST(v_viewer, p_owner_profile_id)
                          AND r.pair_high = GREATEST(v_viewer, p_owner_profile_id)));
    IF FOUND THEN RETURN; END IF;
  END IF;
  INSERT INTO security_event (event_type, actor_type, actor_id, endpoint, target_id, reason)
  VALUES ('BIODATA_DENIED',
          CASE WHEN v_role = 'MEMBER' THEN 'MEMBER' WHEN v_role LIKE '%ADMIN' THEN 'ADMIN' ELSE 'ANONYMOUS' END,
          coalesce(v_member, nullif(current_setting('app.admin_id', true), '')::uuid),
          coalesce(current_setting('app.endpoint', true), 'samaj.get_biodata'),
          p_owner_profile_id::text, 'No accepted biodata request for this pair');
END $$;

-- Write path: a member can only write the biodata of their own profile.
CREATE FUNCTION samaj.upsert_biodata(p_payload_enc bytea, p_key_id varchar, p_schema_version smallint)
RETURNS void LANGUAGE plpgsql SECURITY DEFINER SET search_path = samaj, pg_temp AS $$
DECLARE v_profile uuid;
BEGIN
  IF current_setting('app.actor_role', true) IS DISTINCT FROM 'MEMBER' THEN
    RAISE EXCEPTION 'only members can write biodata' USING ERRCODE = '42501';
  END IF;
  SELECT profile_id INTO STRICT v_profile FROM matrimony_profile
   WHERE member_id = current_setting('app.member_id')::uuid;
  INSERT INTO biodata (profile_id, payload_enc, key_id, schema_version)
  VALUES (v_profile, p_payload_enc, p_key_id, p_schema_version)
  ON CONFLICT (profile_id) DO UPDATE
     SET payload_enc = EXCLUDED.payload_enc, key_id = EXCLUDED.key_id,
         schema_version = EXCLUDED.schema_version;
END $$;

REVOKE ALL ON FUNCTION samaj.get_biodata(uuid), samaj.upsert_biodata(bytea, varchar, smallint) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION samaj.get_biodata(uuid), samaj.upsert_biodata(bytea, varchar, smallint) TO samaj_app;
```

City Scope (Row-Level Security)

```
-- City scope for City Admins, enforced in the database as well as the API (FR-71).
-- The API sets these per transaction:  SELECT set_config('app.actor_role','CITY_ADMIN',true),
--                                             set_config('app.city_id', :city, true);
ALTER TABLE samaj.member ENABLE ROW LEVEL SECURITY;
CREATE POLICY city_scope ON samaj.member TO samaj_app
  USING (current_setting('app.actor_role', true) IN ('MEMBER', 'SUPER_ADMIN')
         OR city_id = nullif(current_setting('app.city_id', true), '')::uuid);
CREATE POLICY background ON samaj.member TO samaj_worker, samaj_jobs USING (true);
-- Repeated for household, family_member, identity_document, content_item, import_job, export_job.
-- If app.actor_role is not set, the policy matches nothing (default deny).
```

Data Encryption

Layer	Control
At rest	RDS storage encryption (AES-256, customer-managed KMS key). Snapshots, replicas and cross-region copies inherit it. Object storage uses SSE-KMS on private buckets, served through pre-signed URLs that expire in 15 minutes or less.
Field level	`biodata.payload_enc`, `member.mobile_enc`, `member.address_enc`, `admin_user.totp_secret_enc` and `import_row.payload_enc` use **AES-256-GCM** envelope encryption in the API with KMS data keys. The database never sees a key. Biodata uses a separate KMS key, which only the API's biodata path may use to decrypt.
Lookup hashing	`member.mobile_hash` and `otp_challenge.mobile_hash` are HMAC-SHA256 over the E.164 number with a secret pepper in Secrets Manager. Equality search works without decryption, and the hashes resist rainbow tables.
Secrets	Argon2id for admin passwords and OTP codes. SHA-256 for refresh tokens. Export passwords are never stored or logged (FR-85).
In transit	TLS 1.2+ on every hop: app to ALB, ALB to API, API to PgBouncer, and PgBouncer to RDS. `rds.force_ssl = 1`, and clients verify the RDS CA (`sslmode=verify-full`).

Regulatory Compliance

DPDP ACT 2023 (INDIA)

- **Consent:** a member row is created only together with a `consent` row for each purpose and notice version (FR-07). Withdrawing consent creates a `data_rights_request` of type `CONSENT_WITHDRAWAL`, which starts erasure (FR-99).
- **Rights:** access, correction, erasure and grievance requests are tracked in `data_rights_request` and `feedback_report`, with a 30-day `due_at` deadline and an overdue alert.
- **Minimisation:** staff troubleshooting sees members only through a masking view. Audit JSON stores changed field names with masked values. Exports use a column allow-list that has no biodata (FR-84).
- **Retention limits:** archive at 12 months and purge at 24 months (SOP §8). ID documents are deleted at 90 days, and import payloads are wiped at 30 days.
- **Breach readiness:** `security_event`, `audit_log` and CloudTrail support the investigation and notification timelines. Data stays in India regions (ASM-11).

```
CREATE VIEW samaj.member_masked AS
SELECT member_id, city_id, household_id, status, gender,
       date_part('year', age(dob))::int  AS age_years,
       '••••••' || mobile_last4          AS mobile_masked,
       left(full_name, 1) || '•••'       AS name_masked,
       created_at, last_login_at, archived_at, deleted_at, erased_at
  FROM samaj.member;
GRANT SELECT ON samaj.member_masked TO samaj_ops_ro;
```

Append-only Audit & Security Logs

FR-87 requires that no user, including the Super Admin, can change audit entries. Privileges are revoked and a trigger rejects any `UPDATE` or `DELETE`, even from a role that was granted one by mistake.

```
REVOKE UPDATE, DELETE, TRUNCATE ON samaj.audit_log, samaj.security_event
  FROM PUBLIC, samaj_app, samaj_worker, samaj_jobs, samaj_ops_ro, samaj_auditor;

CREATE FUNCTION samaj.block_log_change() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN RAISE EXCEPTION '% is append-only', TG_TABLE_NAME USING ERRCODE = '42501'; END $$;

CREATE TRIGGER trg_audit_append_only BEFORE UPDATE OR DELETE ON samaj.audit_log
  FOR EACH ROW EXECUTE FUNCTION samaj.block_log_change();
CREATE TRIGGER trg_security_append_only BEFORE UPDATE OR DELETE ON samaj.security_event
  FOR EACH ROW EXECUTE FUNCTION samaj.block_log_change();
-- Retention removes whole monthly partitions (DROP), which the owner role does after the S3 export.
```

7 Operations, Backup & Disaster Recovery

Backup Schedule

Backup	Frequency	Retention	Where
Automated snapshot (full)	Daily, 19:30–20:00 UTC (01:00–01:30 IST, low traffic)	14 days	RDS, ap-south-1
Transaction log (continuous)	WAL archived every 5 minutes	14 days (PITR window)	RDS, ap-south-1
Pre-release snapshot	Before every 02:00 IST deployment (SOP §10)	30 days	RDS manual snapshot
Monthly snapshot	1st of each month, AWS Backup	12 months	Copied to ap-south-2 (Hyderabad), which keeps data in India
Media	S3 versioning + cross-region replication	Non-current versions 30 days	ap-south-2

RTO & RPO

RPO

≤ 5 min

Point-in-time recovery from 5-minute WAL archives. The SRS target is ≤ 24 h.

RTO: AZ FAILURE

≤ 2 min

Automatic Multi-AZ failover; PgBouncer reconnects

RTO: REGION LOSS

≤ 4 h

Restore the latest cross-region snapshot in ap-south-2 (NFR-10)

High Availability

The primary runs Multi-AZ with a synchronous standby. Failover is automatic and the endpoint name does not change. The read replica is asynchronous and is **not** a write failover target. Reads that must be fresh, such as biodata, request status and approval queues, always go to the primary. A **quarterly restore drill** restores the latest snapshot to a scratch instance (NFR-10). It then runs row counts, the constraint checks and the automated biodata privacy suite against the restored data before the instance is destroyed.

Monitoring & Alerts

Signal	Threshold	Action
Database unreachable (ERR_DB_CON)	Health check fails for 2 minutes	Page on-call (FR-90). Check RDS events and the failover status.
CPU utilisation	> 80% for 10 min	Alert. Review `pg_stat_statements` top queries.
Outbox backlog	Oldest QUEUED row > 30 s	Alert. The push SLA (NFR-05) is at risk; scale the workers.
Panchang missing	No valid `panchang_day` for today at 04:00 IST	Raise ERR_API_PN to the City Admin by push and e-mail (FR-60).
Approval queue age	PENDING member older than 72 h	Escalate to the Super Admin (FR-16).
Biodata denials	> 20 `BIODATA_DENIED` per hour, or any from an ADMIN session	Security alert: possible scraping, or a code path missing its check.

Signal	Threshold	Action
Rights requests overdue	`due_at` passed and not completed	Alert the grievance officer (DPDP).
Replica lag	> 60 s	Alert. The audit view shows a "data delayed" banner.
Partition premake	< 2 future partitions	Page, because inserts into the logs would fail at the month boundary.
Free storage	< 20%	Alert. Confirm the partition drops and purge jobs ran.

8 Appendix

A. Generated DDL

Generated from the same schema definition as the data dictionary, and tested on PostgreSQL 16. Partitions, roles, grants, the biodata functions, RLS policies, the masking view and the append-only triggers (§5–§6) are applied by Alembic migrations after these statements.

DDL · PostgreSQL 16

```
CREATE SCHEMA IF NOT EXISTS samaj;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS citext;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE SEQUENCE IF NOT EXISTS samaj.audit_log_audit_id_seq AS BIGINT;
CREATE SEQUENCE IF NOT EXISTS samaj.security_event_event_id_seq AS BIGINT;
CREATE OR REPLACE FUNCTION samaj.set_updated_at() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$;

-- City: A city served by the deployment. Every member, household, City Admin and city-targeted content item
belongs to exactly one city.
CREATE TABLE samaj.city (
    city_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name        VARCHAR(80) NOT NULL,
    state       VARCHAR(60) NOT NULL,
    is_active   BOOLEAN NOT NULL DEFAULT true,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uq_city_name_state UNIQUE (name, state)
);

-- Admin User: A Super Admin or City Admin who signs in to the Admin Dashboard with password + TOTP. Exactly one
active City Admin per city (FR-70).
CREATE TABLE samaj.admin_user (
    admin_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role             VARCHAR(12) NOT NULL CHECK (role IN ('SUPER_ADMIN','CITY_ADMIN')),
    city_id          UUID,
    full_name        VARCHAR(120) NOT NULL,
    email            CITEXT NOT NULL UNIQUE,
    password_hash    VARCHAR(255) NOT NULL,
    totp_secret_enc  BYTEA,
    mfa_enrolled_at  TIMESTAMPTZ,
    status           VARCHAR(20) NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','SUSPENDED','DEACTIVATED')),
    last_login_at    TIMESTAMPTZ,
    deactivated_at   TIMESTAMPTZ,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_admin_user_city_id FOREIGN KEY (city_id) REFERENCES samaj.city (city_id) ON DELETE RESTRICT,
    CONSTRAINT ck_admin_user_1 CHECK ((role = 'SUPER_ADMIN' AND city_id IS NULL) OR (role = 'CITY_ADMIN' AND
city_id IS NOT NULL))
);
CREATE UNIQUE INDEX ux_admin_user_one_city_admin ON samaj.admin_user (city_id) WHERE role = 'CITY_ADMIN' AND
status = 'ACTIVE';
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.admin_user FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Household: A family unit in the directory. Members link to it; approved non-app relatives are family_member
rows under it.
CREATE TABLE samaj.household (
    household_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    city_id       UUID NOT NULL,
    family_name   VARCHAR(120) NOT NULL,
    native_place  VARCHAR(120),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_household_city_id FOREIGN KEY (city_id) REFERENCES samaj.city (city_id) ON DELETE RESTRICT
);
CREATE INDEX ix_household_city ON samaj.household (city_id);
CREATE INDEX ix_household_family_name_trgm ON samaj.household USING gin (family_name gin_trgm_ops);
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.household FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Member: A registered app user, identified by a unique mobile number. Lifecycle: PENDING → ACTIVE / REJECTED →
ARCHIVED → purged (anonymised).
CREATE TABLE samaj.member (
    member_id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    city_id               UUID NOT NULL,
    household_id          UUID,
```

```
    is_household_head     BOOLEAN NOT NULL DEFAULT false,
    mobile_enc            BYTEA,
    mobile_hash           BYTEA NOT NULL UNIQUE,
    mobile_last4          CHAR(4),
    full_name             VARCHAR(120),
    gender                VARCHAR(10) CHECK (gender IN ('MALE','FEMALE','OTHER')),
    dob                   DATE,
    address_enc           BYTEA,
    status                VARCHAR(20) NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING','ACTIVE','REJECTED','ARCHIVED')),
    rejection_reason      VARCHAR(500),
    reviewed_by_admin_id  UUID,
    reviewed_at           TIMESTAMPTZ,
    hide_phone            BOOLEAN NOT NULL DEFAULT false,
    hide_address          BOOLEAN NOT NULL DEFAULT false,
    notify_news           BOOLEAN NOT NULL DEFAULT true,
    source                VARCHAR(10) NOT NULL DEFAULT 'APP' CHECK (source IN ('APP','IMPORT')),
    last_login_at         TIMESTAMPTZ,
    archive_notice_at     TIMESTAMPTZ,
    archived_at           TIMESTAMPTZ,
    deleted_at            TIMESTAMPTZ,
    erased_at             TIMESTAMPTZ,
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_member_city_id FOREIGN KEY (city_id) REFERENCES samaj.city (city_id) ON DELETE RESTRICT,
    CONSTRAINT fk_member_household_id FOREIGN KEY (household_id) REFERENCES samaj.household (household_id) ON
DELETE SET NULL,
    CONSTRAINT fk_member_reviewed_by_admin_id FOREIGN KEY (reviewed_by_admin_id) REFERENCES samaj.admin_user
(admin_id) ON DELETE SET NULL,
    CONSTRAINT ck_member_1 CHECK (status <> 'REJECTED' OR rejection_reason IS NOT NULL),
    CONSTRAINT ck_member_2 CHECK (dob <= CURRENT_DATE)
);
CREATE INDEX ix_member_city_status ON samaj.member (city_id, status, created_at);
CREATE INDEX ix_member_household ON samaj.member (household_id);
CREATE UNIQUE INDEX ux_member_one_head ON samaj.member (household_id) WHERE is_household_head;
CREATE INDEX ix_member_name_trgm ON samaj.member USING gin (full_name gin_trgm_ops) WHERE status = 'ACTIVE' AND
deleted_at IS NULL;
CREATE INDEX ix_member_last_login ON samaj.member (last_login_at) WHERE status IN ('ACTIVE','ARCHIVED') AND
erased_at IS NULL;
CREATE INDEX ix_member_reviewed_by ON samaj.member (reviewed_by_admin_id);
CREATE INDEX ix_member_deleted ON samaj.member (deleted_at) WHERE deleted_at IS NOT NULL;
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.member FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Family Member: A relative added to a household through a family-addition request (FR-17). Visible in the
directory only once APPROVED.
CREATE TABLE samaj.family_member (
    family_member_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    household_id            UUID NOT NULL,
    requested_by_member_id  UUID NOT NULL,
    linked_member_id        UUID,
    full_name               VARCHAR(120) NOT NULL,
    relationship            VARCHAR(30) NOT NULL,
    gender                  VARCHAR(10) CHECK (gender IN ('MALE','FEMALE','OTHER')),
    dob                     DATE,
    status                  VARCHAR(20) NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING','APPROVED','REJECTED')),
    rejection_reason        VARCHAR(500),
    reviewed_by_admin_id    UUID,
    reviewed_at             TIMESTAMPTZ,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_family_member_household_id FOREIGN KEY (household_id) REFERENCES samaj.household (household_id)
ON DELETE CASCADE,
    CONSTRAINT fk_family_member_requested_by_member_id FOREIGN KEY (requested_by_member_id) REFERENCES
samaj.member (member_id) ON DELETE RESTRICT,
    CONSTRAINT fk_family_member_linked_member_id FOREIGN KEY (linked_member_id) REFERENCES samaj.member
(member_id) ON DELETE SET NULL,
    CONSTRAINT fk_family_member_reviewed_by_admin_id FOREIGN KEY (reviewed_by_admin_id) REFERENCES
samaj.admin_user (admin_id) ON DELETE SET NULL,
    CONSTRAINT ck_family_member_1 CHECK (status <> 'REJECTED' OR rejection_reason IS NOT NULL)
);
CREATE INDEX ix_family_member_household ON samaj.family_member (household_id) WHERE status = 'APPROVED';
CREATE INDEX ix_family_member_pending ON samaj.family_member (status, created_at) WHERE status = 'PENDING';
CREATE INDEX ix_family_member_requested_by ON samaj.family_member (requested_by_member_id);
CREATE INDEX ix_family_member_linked ON samaj.family_member (linked_member_id);
CREATE INDEX ix_family_member_reviewed_by ON samaj.family_member (reviewed_by_admin_id);

-- Identity Document: Optional ID uploaded at registration (FR-09). The file sits in a private, KMS-encrypted
bucket; this row holds only its key and metadata.
CREATE TABLE samaj.identity_document (
    document_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id             UUID NOT NULL,
    storage_key           VARCHAR(255) NOT NULL,
    mime_type             VARCHAR(40) NOT NULL CHECK (mime_type IN ('image/jpeg','image/png','application/pdf')),
```

```
    size_bytes            INTEGER NOT NULL CHECK (size_bytes BETWEEN 1 AND 2097152),
    sha256                CHAR(64) NOT NULL,
    reviewed_by_admin_id  UUID,
    reviewed_at           TIMESTAMPTZ,
    purge_after           TIMESTAMPTZ,
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_identity_document_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON
DELETE CASCADE,
    CONSTRAINT fk_identity_document_reviewed_by_admin_id FOREIGN KEY (reviewed_by_admin_id) REFERENCES
samaj.admin_user (admin_id) ON DELETE SET NULL
);
CREATE INDEX ix_identity_document_member ON samaj.identity_document (member_id);
CREATE INDEX ix_identity_document_reviewed_by ON samaj.identity_document (reviewed_by_admin_id);
CREATE INDEX ix_identity_document_purge ON samaj.identity_document (purge_after) WHERE purge_after IS NOT NULL;

-- Consent: Purpose-specific DPDP consent captured before a profile is created (FR-07). Append-style: a new notice
version creates a new row.
CREATE TABLE samaj.consent (
    consent_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id       UUID NOT NULL,
    purpose         VARCHAR(30) NOT NULL CHECK (purpose IN
('MEMBERSHIP','MATRIMONY','DIRECTORY','NOTIFICATIONS')),
    notice_version  VARCHAR(20) NOT NULL,
    granted_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    withdrawn_at    TIMESTAMPTZ,
    channel         VARCHAR(10) NOT NULL DEFAULT 'APP' CHECK (channel IN ('APP','IMPORT')),
    CONSTRAINT fk_consent_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON DELETE
RESTRICT,
    CONSTRAINT uq_consent_member_id_purpose_notice_version UNIQUE (member_id, purpose, notice_version)
);
CREATE INDEX ix_consent_member ON samaj.consent (member_id);

-- OTP Challenge: A one-time password sent for login, registration or erasure confirmation. Enforces 5-minute
expiry, 3 attempts and 5 sends/hour (FR-03, FR-04).
CREATE TABLE samaj.otp_challenge (
    otp_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    mobile_hash  BYTEA NOT NULL,
    purpose      VARCHAR(12) NOT NULL CHECK (purpose IN ('LOGIN','REGISTER','ERASURE')),
    code_hash    VARCHAR(255) NOT NULL,
    attempts     SMALLINT NOT NULL DEFAULT 0 CHECK (attempts BETWEEN 0 AND 3),
    expires_at   TIMESTAMPTZ NOT NULL,
    consumed_at  TIMESTAMPTZ,
    request_ip   INET,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT ck_otp_challenge_1 CHECK (expires_at > created_at)
);
CREATE INDEX ix_otp_challenge_mobile_time ON samaj.otp_challenge (mobile_hash, created_at DESC);
CREATE INDEX ix_otp_challenge_created ON samaj.otp_challenge USING brin (created_at);

-- Auth Session: A refresh-token session for a member (30-day rotating refresh, NFR-16) or an admin (30-min idle
timeout, FR-73). Exactly one principal per row.
CREATE TABLE samaj.auth_session (
    session_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id           UUID,
    admin_id            UUID,
    refresh_token_hash  BYTEA NOT NULL UNIQUE,
    mfa_verified        BOOLEAN NOT NULL DEFAULT false,
    device_label        VARCHAR(80),
    last_activity_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at          TIMESTAMPTZ NOT NULL,
    revoked_at          TIMESTAMPTZ,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_auth_session_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON DELETE
CASCADE,
    CONSTRAINT fk_auth_session_admin_id FOREIGN KEY (admin_id) REFERENCES samaj.admin_user (admin_id) ON DELETE
CASCADE,
    CONSTRAINT ck_auth_session_1 CHECK ((member_id IS NULL) <> (admin_id IS NULL)),
    CONSTRAINT ck_auth_session_2 CHECK (admin_id IS NULL OR mfa_verified)
);
CREATE INDEX ix_auth_session_member ON samaj.auth_session (member_id) WHERE revoked_at IS NULL;
CREATE INDEX ix_auth_session_admin ON samaj.auth_session (admin_id) WHERE revoked_at IS NULL;
CREATE INDEX ix_auth_session_expires ON samaj.auth_session (expires_at);

-- Device Token: An FCM registration token for a member's device (FR-53). Marked invalid when FCM reports
UNREGISTERED (FR-55).
CREATE TABLE samaj.device_token (
    device_token_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id        UUID NOT NULL,
    platform         VARCHAR(8) NOT NULL CHECK (platform IN ('ANDROID','IOS')),
    fcm_token        VARCHAR(512) NOT NULL UNIQUE,
    app_version      VARCHAR(20),
    is_valid         BOOLEAN NOT NULL DEFAULT true,
    last_seen_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    invalidated_at   TIMESTAMPTZ,
```

```
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_device_token_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON DELETE
CASCADE
);
CREATE INDEX ix_device_token_member ON samaj.device_token (member_id) WHERE is_valid;
CREATE INDEX ix_device_token_invalidated ON samaj.device_token (invalidated_at) WHERE NOT is_valid;

-- Matrimony Profile: The public summary of a member's matrimony listing (FR-08, FR-20). Contains no biodata —
that lives in the separately protected biodata table.
CREATE TABLE samaj.matrimony_profile (
    profile_id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id              UUID NOT NULL UNIQUE,
    location_city_id       UUID NOT NULL,
    managed_by             VARCHAR(10) NOT NULL DEFAULT 'SELF' CHECK (managed_by IN
('SELF','PARENT','SIBLING','RELATIVE')),
    education_level        VARCHAR(20) NOT NULL CHECK (education_level IN
('SCHOOL','DIPLOMA','GRADUATE','POSTGRADUATE','DOCTORATE','OTHER')),
    education_detail       VARCHAR(120),
    occupation             VARCHAR(80),
    family_background      VARCHAR(20) NOT NULL,
    about                  VARCHAR(500),
    status                 VARCHAR(20) NOT NULL DEFAULT 'DRAFT' CHECK (status IN
('DRAFT','PENDING_MODERATION','APPROVED','REJECTED','ARCHIVED')),
    moderation_reason      VARCHAR(500),
    moderated_by_admin_id  UUID,
    moderated_at           TIMESTAMPTZ,
    created_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_matrimony_profile_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON
DELETE RESTRICT,
    CONSTRAINT fk_matrimony_profile_location_city_id FOREIGN KEY (location_city_id) REFERENCES samaj.city
(city_id) ON DELETE RESTRICT,
    CONSTRAINT fk_matrimony_profile_moderated_by_admin_id FOREIGN KEY (moderated_by_admin_id) REFERENCES
samaj.admin_user (admin_id) ON DELETE SET NULL,
    CONSTRAINT ck_matrimony_profile_1 CHECK (status <> 'REJECTED' OR moderation_reason IS NOT NULL)
);
CREATE INDEX ix_matrimony_profile_moderation ON samaj.matrimony_profile (status, created_at) WHERE status =
'PENDING_MODERATION';
CREATE INDEX ix_matrimony_profile_location ON samaj.matrimony_profile (location_city_id);
CREATE INDEX ix_matrimony_profile_moderated_by ON samaj.matrimony_profile (moderated_by_admin_id);
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.matrimony_profile FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Biodata: The private biodata of a matrimony profile, stored as one AES-256-GCM encrypted JSON document.
Readable only through samaj.get_biodata(), which enforces the accepted-pair rule.
CREATE TABLE samaj.biodata (
    profile_id      UUID PRIMARY KEY,
    payload_enc     BYTEA NOT NULL,
    key_id          VARCHAR(64) NOT NULL,
    schema_version  SMALLINT NOT NULL DEFAULT 1,
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_biodata_profile_id FOREIGN KEY (profile_id) REFERENCES samaj.matrimony_profile (profile_id) ON
DELETE CASCADE
);
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.biodata FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Matrimony Photo: A photo on a matrimony profile, moderated against community guidelines before it is shown
(FR-09, FR-23). Maximum 5 per profile.
CREATE TABLE samaj.matrimony_photo (
    photo_id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id             UUID NOT NULL,
    storage_key            VARCHAR(255) NOT NULL,
    mime_type              VARCHAR(20) NOT NULL CHECK (mime_type IN ('image/jpeg','image/png')),
    size_bytes             INTEGER NOT NULL CHECK (size_bytes BETWEEN 1 AND 2097152),
    sort_order             SMALLINT NOT NULL CHECK (sort_order BETWEEN 1 AND 5),
    status                 VARCHAR(20) NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING','APPROVED','REJECTED')),
    rejection_reason       VARCHAR(500),
    moderated_by_admin_id  UUID,
    moderated_at           TIMESTAMPTZ,
    created_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_matrimony_photo_profile_id FOREIGN KEY (profile_id) REFERENCES samaj.matrimony_profile
(profile_id) ON DELETE CASCADE,
    CONSTRAINT fk_matrimony_photo_moderated_by_admin_id FOREIGN KEY (moderated_by_admin_id) REFERENCES
samaj.admin_user (admin_id) ON DELETE SET NULL,
    CONSTRAINT uq_matrimony_photo_profile_id_sort_order UNIQUE (profile_id, sort_order),
    CONSTRAINT ck_matrimony_photo_1 CHECK (status <> 'REJECTED' OR rejection_reason IS NOT NULL)
);
CREATE INDEX ix_matrimony_photo_pending ON samaj.matrimony_photo (created_at) WHERE status = 'PENDING';
CREATE INDEX ix_matrimony_photo_moderated_by ON samaj.matrimony_photo (moderated_by_admin_id);
CREATE MATERIALIZED VIEW samaj.matrimony_search_mv AS
SELECT p.profile_id, p.member_id, m.gender,
       date_part('year', age(m.dob))::int AS age_years,
```

```
       p.education_level, p.location_city_id, p.family_background,
       p.education_detail, p.occupation,
       (SELECT ph.storage_key FROM samaj.matrimony_photo ph
         WHERE ph.profile_id = p.profile_id AND ph.status = 'APPROVED'
         ORDER BY ph.sort_order LIMIT 1) AS cover_photo_key
  FROM samaj.matrimony_profile p
  JOIN samaj.member m ON m.member_id = p.member_id
 WHERE p.status = 'APPROVED' AND m.status = 'ACTIVE' AND m.deleted_at IS NULL
WITH DATA;
CREATE UNIQUE INDEX ux_matrimony_search_mv_profile ON samaj.matrimony_search_mv (profile_id);
CREATE INDEX ix_matrimony_search_mv_filters ON samaj.matrimony_search_mv (gender, age_years, location_city_id,
education_level, family_background);

-- Biodata Request: User A's request to exchange biodata with User B (SOP §6.2). ACCEPTED grants mutual access to
that pair only. At most one PENDING or ACCEPTED request per pair, in either direction (FR-30).
CREATE TABLE samaj.biodata_request (
    request_id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    requester_profile_id    UUID NOT NULL,
    recipient_profile_id    UUID NOT NULL,
    pair_low                UUID GENERATED ALWAYS AS (LEAST(requester_profile_id, recipient_profile_id)) STORED
NOT NULL,
    pair_high               UUID GENERATED ALWAYS AS (GREATEST(requester_profile_id, recipient_profile_id))
STORED NOT NULL,
    status                  VARCHAR(20) NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING','ACCEPTED','DECLINED','WITHDRAWN')),
    responded_at            TIMESTAMPTZ,
    resend_allowed_at       TIMESTAMPTZ,
    withdrawn_at            TIMESTAMPTZ,
    withdrawn_by_profile_id UUID,
    created_at              TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_biodata_request_requester_profile_id FOREIGN KEY (requester_profile_id) REFERENCES
samaj.matrimony_profile (profile_id) ON DELETE CASCADE,
    CONSTRAINT fk_biodata_request_recipient_profile_id FOREIGN KEY (recipient_profile_id) REFERENCES
samaj.matrimony_profile (profile_id) ON DELETE CASCADE,
    CONSTRAINT fk_biodata_request_withdrawn_by_profile_id FOREIGN KEY (withdrawn_by_profile_id) REFERENCES
samaj.matrimony_profile (profile_id) ON DELETE SET NULL,
    CONSTRAINT ck_biodata_request_1 CHECK (requester_profile_id <> recipient_profile_id),
    CONSTRAINT ck_biodata_request_2 CHECK (status = 'PENDING' OR responded_at IS NOT NULL),
    CONSTRAINT ck_biodata_request_3 CHECK (status <> 'WITHDRAWN' OR (withdrawn_at IS NOT NULL AND
withdrawn_by_profile_id IS NOT NULL))
);
CREATE UNIQUE INDEX ux_biodata_request_open_pair ON samaj.biodata_request (pair_low, pair_high) WHERE status IN
('PENDING','ACCEPTED');
CREATE INDEX ix_biodata_request_recipient ON samaj.biodata_request (recipient_profile_id, status, created_at
DESC);
CREATE INDEX ix_biodata_request_requester ON samaj.biodata_request (requester_profile_id, created_at DESC);
CREATE INDEX ix_biodata_request_declined_pair ON samaj.biodata_request (pair_low, pair_high, resend_allowed_at)
WHERE status = 'DECLINED';
CREATE INDEX ix_biodata_request_withdrawn_by ON samaj.biodata_request (withdrawn_by_profile_id);

-- Content Item: A news item or ad campaign scheduled in Content Management (SOP §6.3). Goes live at start_at and
expires at end_at automatically (FR-47).
CREATE TABLE samaj.content_item (
    content_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    content_type        VARCHAR(4) NOT NULL CHECK (content_type IN ('NEWS','AD')),
    city_id             UUID,
    title               VARCHAR(200) NOT NULL,
    body                TEXT,
    media_key           VARCHAR(255),
    creative_format     VARCHAR(10) CHECK (creative_format IN ('BANNER','FULLSCREEN')),
    media_width         SMALLINT,
    media_height        SMALLINT,
    start_at            TIMESTAMPTZ NOT NULL,
    end_at              TIMESTAMPTZ NOT NULL,
    status              VARCHAR(20) NOT NULL DEFAULT 'DRAFT' CHECK (status IN
('DRAFT','SCHEDULED','LIVE','EXPIRED','CANCELLED')),
    created_by_admin_id UUID NOT NULL,
    published_at        TIMESTAMPTZ,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_content_item_city_id FOREIGN KEY (city_id) REFERENCES samaj.city (city_id) ON DELETE RESTRICT,
    CONSTRAINT fk_content_item_created_by_admin_id FOREIGN KEY (created_by_admin_id) REFERENCES samaj.admin_user
(admin_id) ON DELETE RESTRICT,
    CONSTRAINT ck_content_item_1 CHECK (end_at > start_at),
    CONSTRAINT ck_content_item_2 CHECK (content_type = 'NEWS' OR (creative_format IS NOT NULL AND media_key IS NOT
NULL)),
    CONSTRAINT ck_content_item_3 CHECK (creative_format IS DISTINCT FROM 'FULLSCREEN' OR (media_width = 1080 AND
media_height = 1920)),
    CONSTRAINT ck_content_item_4 CHECK (content_type = 'AD' OR body IS NOT NULL)
);
CREATE INDEX ix_content_item_feed ON samaj.content_item (status, city_id, start_at DESC) WHERE status = 'LIVE';
CREATE INDEX ix_content_item_due ON samaj.content_item (start_at) WHERE status = 'SCHEDULED';
CREATE INDEX ix_content_item_expiring ON samaj.content_item (end_at) WHERE status = 'LIVE';
CREATE INDEX ix_content_item_city ON samaj.content_item (city_id);
```

```
CREATE INDEX ix_content_item_created_by ON samaj.content_item (created_by_admin_id);
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.content_item FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Layout Setting: Key/value configuration for ad dimensions and feed placement (FR-50, FR-51), e.g.
ad_every_n_items = 5.
CREATE TABLE samaj.layout_setting (
    setting_key          VARCHAR(60) PRIMARY KEY,
    value                JSONB NOT NULL,
    updated_by_admin_id  UUID,
    updated_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_layout_setting_updated_by_admin_id FOREIGN KEY (updated_by_admin_id) REFERENCES samaj.admin_user
(admin_id) ON DELETE SET NULL
);
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.layout_setting FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Panchang: The effective Tithi, Nakshatra and Yoga for one date and reference location (SOP §6.4). A manual
entry overrides API values (FR-62).
CREATE TABLE samaj.panchang_day (
    panchang_id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    panchang_date             DATE NOT NULL,
    location_code             VARCHAR(20) NOT NULL DEFAULT 'DEFAULT',
    tithi                     VARCHAR(40) NOT NULL,
    nakshatra                 VARCHAR(40) NOT NULL,
    yoga                      VARCHAR(40) NOT NULL,
    source                    VARCHAR(6) NOT NULL CHECK (source IN ('API','MANUAL')),
    verified_against_calendar BOOLEAN NOT NULL DEFAULT false,
    edited_by_admin_id        UUID,
    fetched_at                TIMESTAMPTZ,
    published_at              TIMESTAMPTZ,
    updated_at                TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_panchang_day_edited_by_admin_id FOREIGN KEY (edited_by_admin_id) REFERENCES samaj.admin_user
(admin_id) ON DELETE SET NULL,
    CONSTRAINT uq_panchang_day_location_code_panchang_date UNIQUE (location_code, panchang_date),
    CONSTRAINT ck_panchang_day_1 CHECK (source = 'API' OR (edited_by_admin_id IS NOT NULL AND
verified_against_calendar))
);
CREATE INDEX ix_panchang_day_edited_by ON samaj.panchang_day (edited_by_admin_id);
CREATE TRIGGER trg_set_updated_at BEFORE UPDATE ON samaj.panchang_day FOR EACH ROW EXECUTE FUNCTION
samaj.set_updated_at();

-- Panchang Revision: Append-only history of every API fetch and manual edit of a Panchang day: the edit history
shown in the dashboard (FR-62).
CREATE TABLE samaj.panchang_revision (
    revision_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    panchang_id         UUID NOT NULL,
    tithi               VARCHAR(40) NOT NULL,
    nakshatra           VARCHAR(40) NOT NULL,
    yoga                VARCHAR(40) NOT NULL,
    source              VARCHAR(6) NOT NULL CHECK (source IN ('API','MANUAL')),
    edited_by_admin_id  UUID,
    api_status          VARCHAR(20),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_panchang_revision_panchang_id FOREIGN KEY (panchang_id) REFERENCES samaj.panchang_day
(panchang_id) ON DELETE RESTRICT,
    CONSTRAINT fk_panchang_revision_edited_by_admin_id FOREIGN KEY (edited_by_admin_id) REFERENCES
samaj.admin_user (admin_id) ON DELETE SET NULL
);
CREATE INDEX ix_panchang_revision_day ON samaj.panchang_revision (panchang_id, created_at DESC);
CREATE INDEX ix_panchang_revision_edited_by ON samaj.panchang_revision (edited_by_admin_id);

-- Notification: Transactional outbox for push / SMS / email. Written in the same transaction as the business
change, then delivered by the worker (≤ 30 s target, NFR-05).
CREATE TABLE samaj.notification_outbox (
    notification_id  UUID NOT NULL DEFAULT gen_random_uuid(),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
    kind             VARCHAR(30) NOT NULL CHECK (kind IN
('ACCOUNT_DECISION','NEWS_LIVE','BIODATA_REQUEST','BIODATA_RESPONSE','FAMILY_DECISION','PROFILE_DECISION','PANCHAN
G_ALERT','ARCHIVE_NOTICE','ESCALATION','IMPORT_REPORT')),
    channel          VARCHAR(6) NOT NULL CHECK (channel IN ('PUSH','SMS','EMAIL')),
    audience         VARCHAR(10) NOT NULL CHECK (audience IN ('MEMBER','CITY','ALL','ADMIN')),
    member_id        UUID,
    city_id          UUID,
    admin_id         UUID,
    content_id       UUID,
    payload          JSONB NOT NULL DEFAULT '{}'::jsonb,
    state            VARCHAR(10) NOT NULL DEFAULT 'QUEUED' CHECK (state IN
('QUEUED','SENDING','SENT','PARTIAL','FAILED')),
    attempts         SMALLINT NOT NULL DEFAULT 0,
    sent_at          TIMESTAMPTZ,
    success_count    INTEGER,
    failure_count    INTEGER,
    CONSTRAINT pk_notification_outbox PRIMARY KEY (notification_id, created_at),
```

```
    CONSTRAINT fk_notification_outbox_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON
DELETE CASCADE,
    CONSTRAINT fk_notification_outbox_city_id FOREIGN KEY (city_id) REFERENCES samaj.city (city_id) ON DELETE
RESTRICT,
    CONSTRAINT fk_notification_outbox_admin_id FOREIGN KEY (admin_id) REFERENCES samaj.admin_user (admin_id) ON
DELETE RESTRICT,
    CONSTRAINT fk_notification_outbox_content_id FOREIGN KEY (content_id) REFERENCES samaj.content_item
(content_id) ON DELETE SET NULL,
    CONSTRAINT ck_notification_outbox_1 CHECK (member_id IS NOT NULL OR audience <> 'MEMBER')
)
PARTITION BY RANGE (created_at);
CREATE INDEX ix_notification_outbox_queue ON samaj.notification_outbox (state, created_at) WHERE state IN
('QUEUED','SENDING');
CREATE INDEX ix_notification_outbox_member ON samaj.notification_outbox (member_id);
CREATE INDEX ix_notification_outbox_content ON samaj.notification_outbox (content_id);
CREATE INDEX ix_notification_outbox_city ON samaj.notification_outbox (city_id);
CREATE INDEX ix_notification_outbox_admin ON samaj.notification_outbox (admin_id);

-- Import Job: A CSV/XLSX member-list upload (SOP §6.5). Validated → dry-run report → committed. A header
mismatch rejects the file before any write (FR-76).
CREATE TABLE samaj.import_job (
    import_job_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    city_id               UUID NOT NULL,
    uploaded_by_admin_id  UUID NOT NULL,
    file_key              VARCHAR(255) NOT NULL,
    file_format           VARCHAR(4) NOT NULL CHECK (file_format IN ('CSV','XLSX')),
    template_version      VARCHAR(20) NOT NULL,
    status                VARCHAR(20) NOT NULL DEFAULT 'VALIDATING' CHECK (status IN
('VALIDATING','REJECTED','DRY_RUN_READY','COMMITTING','COMMITTED','FAILED')),
    total_rows            INTEGER CHECK (total_rows <= 50000),
    new_count             INTEGER NOT NULL DEFAULT 0,
    updated_count         INTEGER NOT NULL DEFAULT 0,
    conflict_count        INTEGER NOT NULL DEFAULT 0,
    error_count           INTEGER NOT NULL DEFAULT 0,
    report_key            VARCHAR(255),
    committed_at          TIMESTAMPTZ,
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_import_job_city_id FOREIGN KEY (city_id) REFERENCES samaj.city (city_id) ON DELETE RESTRICT,
    CONSTRAINT fk_import_job_uploaded_by_admin_id FOREIGN KEY (uploaded_by_admin_id) REFERENCES samaj.admin_user
(admin_id) ON DELETE RESTRICT
);
CREATE INDEX ix_import_job_city_time ON samaj.import_job (city_id, created_at DESC);
CREATE INDEX ix_import_job_uploaded_by ON samaj.import_job (uploaded_by_admin_id);

-- Import Row: One validated row of an import, with its dedup outcome (NEW / UPDATE / CONFLICT / ERROR). Staged
data is encrypted and wiped 30 days after commit.
CREATE TABLE samaj.import_row (
    import_job_id      UUID NOT NULL,
    row_number         INTEGER NOT NULL,
    outcome            VARCHAR(8) NOT NULL CHECK (outcome IN ('NEW','UPDATE','CONFLICT','ERROR')),
    mobile_hash        BYTEA,
    matched_member_id  UUID,
    payload_enc        BYTEA,
    error_message      VARCHAR(500),
    resolution         VARCHAR(10) CHECK (resolution IN ('APPLIED','SKIPPED')),
    CONSTRAINT pk_import_row PRIMARY KEY (import_job_id, row_number),
    CONSTRAINT fk_import_row_import_job_id FOREIGN KEY (import_job_id) REFERENCES samaj.import_job (import_job_id)
ON DELETE CASCADE,
    CONSTRAINT fk_import_row_matched_member_id FOREIGN KEY (matched_member_id) REFERENCES samaj.member (member_id)
ON DELETE SET NULL
);
CREATE INDEX ix_import_row_conflicts ON samaj.import_row (import_job_id) WHERE outcome = 'CONFLICT' AND resolution
IS NULL;
CREATE INDEX ix_import_row_member ON samaj.import_row (matched_member_id);

-- Export Job: Record of an 'Export Filtered' (FR-82). The password and the file are never stored; only who
exported what, and how many rows.
CREATE TABLE samaj.export_job (
    export_job_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    city_id               UUID,
    exported_by_admin_id  UUID NOT NULL,
    filters               JSONB NOT NULL,
    columns_exported      TEXT[] NOT NULL,
    row_count             INTEGER NOT NULL CHECK (row_count >= 0),
    file_format           VARCHAR(4) NOT NULL CHECK (file_format IN ('CSV','XLSX')),
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_export_job_city_id FOREIGN KEY (city_id) REFERENCES samaj.city (city_id) ON DELETE RESTRICT,
    CONSTRAINT fk_export_job_exported_by_admin_id FOREIGN KEY (exported_by_admin_id) REFERENCES samaj.admin_user
(admin_id) ON DELETE RESTRICT
);
CREATE INDEX ix_export_job_admin_time ON samaj.export_job (exported_by_admin_id, created_at DESC);
CREATE INDEX ix_export_job_city ON samaj.export_job (city_id);
```

```
-- Audit Log: Append-only record of every admin action and data-principal event (FR-86, FR-87). No FKs, so log rows outlive the objects they describe.
CREATE TABLE samaj.audit_log (
    audit_id      BIGINT NOT NULL DEFAULT nextval('samaj.audit_log_audit_id_seq'),
    occurred_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    actor_type    VARCHAR(8) NOT NULL CHECK (actor_type IN ('ADMIN','MEMBER','SYSTEM')),
    actor_id      UUID,
    action        VARCHAR(40) NOT NULL,
    target_table  VARCHAR(40),
    target_id     VARCHAR(64),
    city_id       UUID,
    before_state  JSONB,
    after_state   JSONB,
    source_ip     INET,
    request_id    UUID,
    CONSTRAINT pk_audit_log PRIMARY KEY (audit_id, occurred_at)
)
PARTITION BY RANGE (occurred_at);
CREATE INDEX ix_audit_log_occurred ON samaj.audit_log USING brin (occurred_at);
CREATE INDEX ix_audit_log_actor ON samaj.audit_log (actor_id, occurred_at DESC);
CREATE INDEX ix_audit_log_city_action ON samaj.audit_log (city_id, action, occurred_at DESC);
CREATE INDEX ix_audit_log_target ON samaj.audit_log (target_table, target_id);

-- Security Event: Denied or suspicious access, above all every denied biodata read (FR-37), plus OTP lockouts, MFA failures (ERR_AUTH_01) and cross-city 403s.
CREATE TABLE samaj.security_event (
    event_id     BIGINT NOT NULL DEFAULT nextval('samaj.security_event_event_id_seq'),
    occurred_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    event_type   VARCHAR(30) NOT NULL CHECK (event_type IN
('BIODATA_DENIED','CROSS_CITY_DENIED','ID_DOC_DENIED','OTP_LOCKED','OTP_RATE_LIMIT','MFA_FAILED','TOKEN_REUSE')),
    actor_type   VARCHAR(8) NOT NULL CHECK (actor_type IN ('ADMIN','MEMBER','ANONYMOUS')),
    actor_id     UUID,
    endpoint     VARCHAR(120) NOT NULL,
    target_id    VARCHAR(64),
    reason       VARCHAR(200),
    source_ip    INET,
    CONSTRAINT pk_security_event PRIMARY KEY (event_id, occurred_at)
)
PARTITION BY RANGE (occurred_at);
CREATE INDEX ix_security_event_occurred ON samaj.security_event USING brin (occurred_at);
CREATE INDEX ix_security_event_type ON samaj.security_event (event_type, occurred_at DESC);
CREATE INDEX ix_security_event_actor ON samaj.security_event (actor_id, occurred_at DESC);

-- Feedback / Report: In-app feedback, a profile report (FR-91) or a DPDP grievance (FR-100), reviewed daily in the dashboard.
CREATE TABLE samaj.feedback_report (
    feedback_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id            UUID,
    category             VARCHAR(16) NOT NULL CHECK (category IN ('FEEDBACK','PROFILE_REPORT','GRIEVANCE')),
    target_profile_id    UUID,
    message              VARCHAR(2000) NOT NULL,
    status               VARCHAR(20) NOT NULL DEFAULT 'OPEN' CHECK (status IN
('OPEN','IN_REVIEW','RESOLVED','DISMISSED')),
    handled_by_admin_id  UUID,
    resolved_at          TIMESTAMPTZ,
    created_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_feedback_report_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON DELETE
SET NULL,
    CONSTRAINT fk_feedback_report_target_profile_id FOREIGN KEY (target_profile_id) REFERENCES
samaj.matrimony_profile (profile_id) ON DELETE SET NULL,
    CONSTRAINT fk_feedback_report_handled_by_admin_id FOREIGN KEY (handled_by_admin_id) REFERENCES
samaj.admin_user (admin_id) ON DELETE SET NULL,
    CONSTRAINT ck_feedback_report_1 CHECK (category <> 'PROFILE_REPORT' OR target_profile_id IS NOT NULL)
);
CREATE INDEX ix_feedback_report_open ON samaj.feedback_report (category, created_at) WHERE status IN
('OPEN','IN_REVIEW');
CREATE INDEX ix_feedback_report_member ON samaj.feedback_report (member_id);
CREATE INDEX ix_feedback_report_target ON samaj.feedback_report (target_profile_id);
CREATE INDEX ix_feedback_report_handler ON samaj.feedback_report (handled_by_admin_id);

-- Data Rights Request: A DPDP data-principal request: access copy, erasure (OTP-confirmed) or consent withdrawal, which triggers erasure. Completed within 30 days (FR-98, FR-99).
CREATE TABLE samaj.data_rights_request (
    rights_request_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id            UUID NOT NULL,
    request_type         VARCHAR(20) NOT NULL CHECK (request_type IN
('ACCESS','ERASURE','CONSENT_WITHDRAWAL','RETENTION_PURGE')),
    status               VARCHAR(20) NOT NULL DEFAULT 'RECEIVED' CHECK (status IN
('RECEIVED','OTP_CONFIRMED','IN_PROGRESS','COMPLETED','REJECTED')),
    otp_confirmed_at     TIMESTAMPTZ,
    requested_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    due_at               TIMESTAMPTZ NOT NULL DEFAULT now() + INTERVAL '30 days',
    completed_at         TIMESTAMPTZ,
    handled_by_admin_id  UUID,
```

```
    CONSTRAINT fk_data_rights_request_member_id FOREIGN KEY (member_id) REFERENCES samaj.member (member_id) ON
DELETE RESTRICT,
    CONSTRAINT fk_data_rights_request_handled_by_admin_id FOREIGN KEY (handled_by_admin_id) REFERENCES
samaj.admin_user (admin_id) ON DELETE SET NULL,
    CONSTRAINT ck_data_rights_request_1 CHECK (status <> 'COMPLETED' OR completed_at IS NOT NULL)
);
CREATE INDEX ix_data_rights_request_due ON samaj.data_rights_request (due_at) WHERE status NOT IN
('COMPLETED','REJECTED');
CREATE INDEX ix_data_rights_request_member ON samaj.data_rights_request (member_id);
CREATE INDEX ix_data_rights_request_handler ON samaj.data_rights_request (handled_by_admin_id);
```

B. Open Questions

ID	Question	Schema impact	Owner
OQ-02	Can any admin ever view biodata, e.g. for an abuse investigation? Is withdrawal (FR-41) required?	Today `get_biodata()` denies every admin. A break-glass path would need a dual-approval table and audit.	Community leadership
OQ-04	Which reference location(s) does the Panchang use?	`location_code` already allows several; the app would need a location setting.	Super Admin
OQ-06	AWS or Azure, and which India region?	None on the schema. Changes the hosting, backup and secrets sections.	Vrattiks / Client
OQ-08	Which directory fields are visible to all members by default?	May add more `hide_*` flags on `member`.	Community leadership
OQ-10	How long are identity documents kept after approval? (assumed 90 days)	Changes `purge_after`.	Data Protection Contact
OQ-11	Background job stack: Celery + Redis, or ARQ / APScheduler?	None, because the outbox pattern is queue-agnostic.	Vrattiks Architect
OQ-12	Is the biodata field list fixed, or configurable by the Super Admin?	Fixed fields would allow typed columns. Today they are an encrypted JSON document with `schema_version`.	Community leadership
OQ-13	Should search be limited to the opposite gender, and should family background use a fixed value list?	Would add `CHECK` lists and a fixed filter on the search view.	Product
OQ-14	How long are archived audit partitions kept in S3 Object Lock beyond the 24 months online?	Sets the archive retention only.	Super Admin / Legal

C. Risks

Risk	Impact	Mitigation
An API code path returns biodata to a third party	High	No table privilege, so only `get_biodata()` can read it. Denials are logged in the database. The privacy test suite is a release gate, and denials from admin sessions raise alerts.
The `mobile_hash` pepper is lost or leaked	High	Versioned in Secrets Manager. The rotation plan re-hashes in batches from `mobile_enc`.
The API forgets to set `app.actor_role`	Medium	RLS and `get_biodata()` default to deny. An integration test runs for each endpoint.
The search view shows a profile that was just hidden	Medium	The search query re-joins the live status, and the view is refreshed straight after any erasure.
Future log partitions are missing	High	pg_partman pre-creates 3 months, with an alert when fewer than 2 remain.
A bad import overwrites member data	Medium	Dry-run first. Conflicts are never auto-applied, every commit is audited, and PITR can restore to just before the commit.

CHANGE CONTROL

All schema changes go through versioned Alembic migrations reviewed by the Lead Database Architect, and are tested in staging (SOP §9) before the 02:00 IST release. A new version of this document is required for any change to a PII or biodata column, a retention period, a role grant, or the biodata access functions.